Module 2: Introduction to Convolutional Neural Networks (CNNs)

Balaji Srinivasan, Ganapathy Krishnamurthi

Wadhwani School of AI , IIT Madras

Outline of this Module

1. Introduction to Vision Modelling
2. Convolutional Neural Networks (CNNs)
3. Famous CNN Architectures
4. Transfer Learning with CNNs

Image as Data

Introduction to Vision Modelling

How are Images interpreted by machines?

How are Images interpreted by machines?

What is a Digital Image?

- A **digital image** is interpreted by a machine not as a visual picture, but as a large grid (or matrix) of numerical data.
- Each entry in this grid corresponds to a *pixel*, the smallest unit of the image.

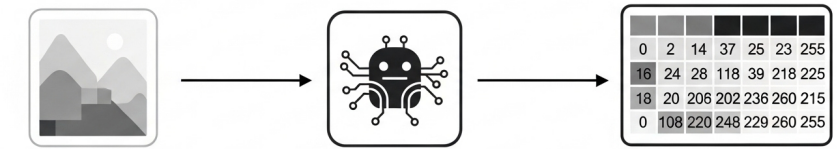


Figure: Illustration showing how an AI interprets an image. (Source:Imagen)

Types of Digital Images - Overview

Types of Digital Images - Overview

Binary Image	Grayscale Image	Color Image (RGB)

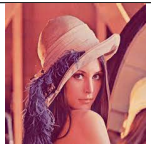
Types of Digital Images - Overview

Binary Image	**Grayscale Image**	**Color Image (RGB)**
Contains only two values, typically representing black and white.	Represents images with shades of gray, ranging from pure black to pure white.	Represents the full spectrum of color by combining different intensities of Red, Green, and Blue light.

Types of Digital Images - Overview

Binary Image	**Grayscale Image**	**Color Image (RGB)**
Contains only two values, typically representing black and white.	Represents images with shades of gray, ranging from pure black to pure white.	Represents the full spectrum of color by combining different intensities of Red, Green, and Blue light.
Each pixel is stored as a single bit: **0** (black) or **1** (white).	Each pixel is a single 8-bit value from **0** (black) to **255** (white).	Each pixel has three 8-bit values, one for each channel **(R, G, B)**

Types of Digital Images - Overview

Binary Image	**Grayscale Image**	**Color Image (RGB)**
Contains only two values, typically representing black and white.	Represents images with shades of gray, ranging from pure black to pure white.	Represents the full spectrum of color by combining different intensities of Red, Green, and Blue light.
Each pixel is stored as a single bit: **0** (black) or **1** (white).	Each pixel is a single 8-bit value from **0** (black) to **255** (white).	Each pixel has three 8-bit values, one for each channel **(R, G, B)**
		

Understanding an RGB Image

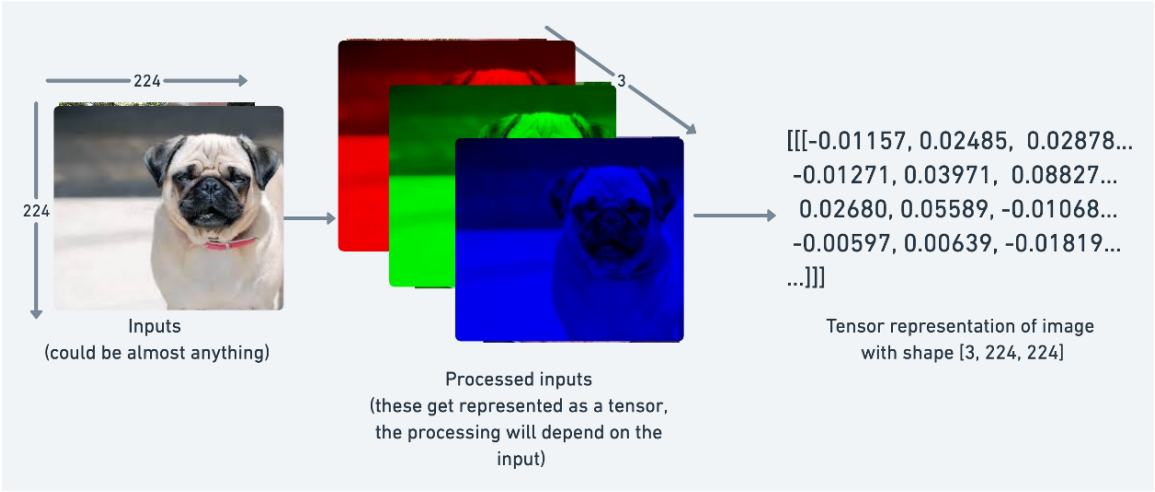
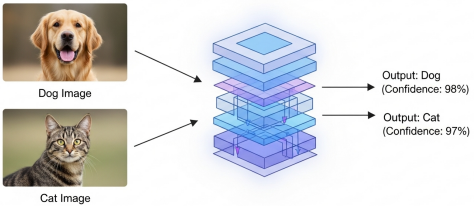


Figure: Illustration of RGB image pixel values

Common Computer Vision Tasks

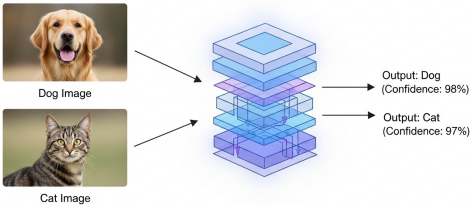
Common Computer Vision Tasks

Image Classification: Assign a single label to an entire image ("cat" or "dog").

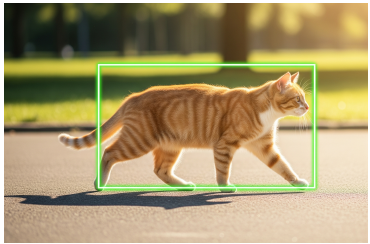


Common Computer Vision Tasks

Image Classification: Assign a single label to an entire image ("cat" or "dog").

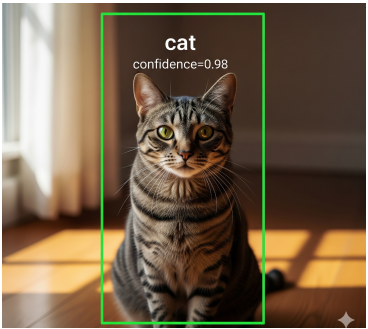


Object detection: Identify objects within an image and draw bounding boxes around them ("there is an object here").



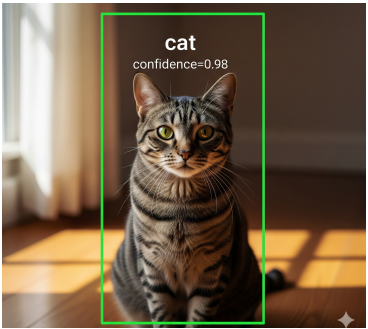
Common Computer Vision Tasks

Object recognition: Detecting the object category as well as its position in image.("There is a cat within this area")

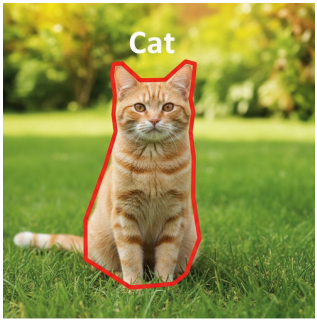


Common Computer Vision Tasks

Object recognition: Detecting the object category as well as its position in image.("There is a cat within this area")



Semantic Segmentation: Assign a class label to every single pixel in the image ("these pixels are cat, these pixels are grass").



Using MLP for Vision Tasks

Using MLP for Vision Tasks

The Naive Approach

How can we apply the Multi-Layer Perceptron (MLP) to an image classification task?

Using MLP for Vision Tasks

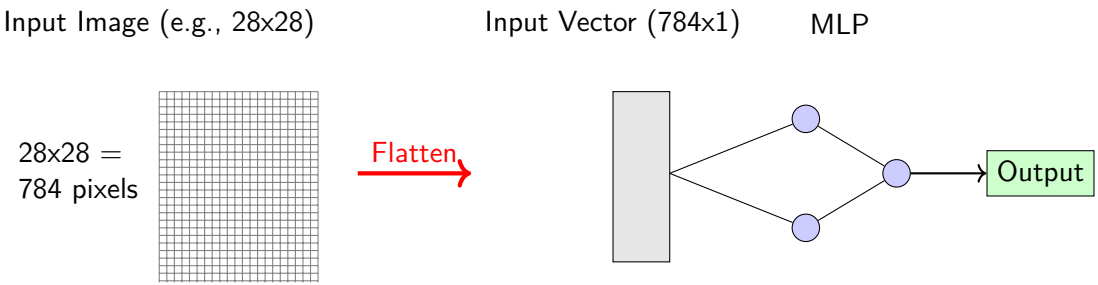
The Naive Approach

How can we apply the Multi-Layer Perceptron (MLP) to an image classification task?
We must convert the 2D/3D image grid into a 1D vector.

Using MLP for Vision Tasks

The Naive Approach

How can we apply the Multi-Layer Perceptron (MLP) to an image classification task?
We must convert the 2D/3D image grid into a 1D vector.



1. **Flatten the image:** Take the grid of pixels and unroll it into a single, long vector.
2. **Feed into MLP:** This vector becomes the input for a standard Fully Connected Network.

Why MLP is not an optimal choice?

Why MLP is not an optimal choice?

While technically possible, using MLPs for images is a terrible idea for several fundamental reasons.

Why MLP is not an optimal choice?

While technically possible, using MLPs for images is a terrible idea for several fundamental reasons.

1. Massive Number of Parameters

Fully connected layers are parameter-hungry.

1. A tiny 100x100 RGB image has:
$$100 \times 100 \times 3 = 30,000 \text{ input features (pixels).}$$
2. If the first hidden layer has just 1000 neurons, this single connection requires:
$$30,000 \times 1000 = \textbf{30 million} \text{ weights!}$$

Why MLP is not an optimal choice?

While technically possible, using MLPs for images is a terrible idea for several fundamental reasons.

1. Massive Number of Parameters

Fully connected layers are parameter-hungry.

1. A tiny 100x100 RGB image has:
$$100 \times 100 \times 3 = 30,000 \text{ input features (pixels).}$$
2. If the first hidden layer has just 1000 neurons, this single connection requires:
$$30,000 \times 1000 = \textbf{30 million} \text{ weights!}$$
3. This leads to extremely **slow training**, **massive memory usage**, and a very high risk of **overfitting**.

Why MLP is not an optimal choice?

2. Destruction of Spatial Structure

- The "flattening" process throws away all spatial information. The model has no idea about which pixels were originally neighbors.

Why MLP is not an optimal choice?

2. Destruction of Spatial Structure

- The "flattening" process throws away all spatial information. The model has no idea about which pixels were originally neighbors.
- A pixel at the top-left is treated the same as a pixel at the bottom-right, even though their spatial relationship is critical for understanding the image.

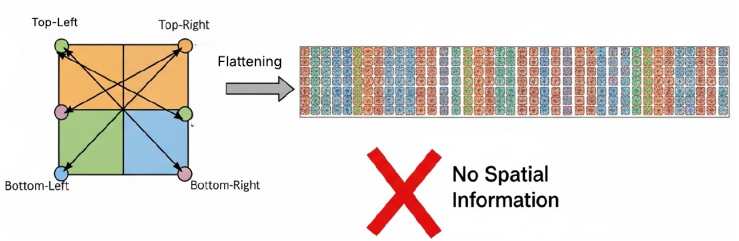


Figure: Adverse effects of image flattening in FC Layers. (source:Imagen)

Why MLP is not an optimal choice?

3. Lack of Translation Invariance

- MLPs are not robust to an object's position.

Why MLP is not an optimal choice?

3. Lack of Translation Invariance

- MLPs are not robust to an object's position.
- The model learns specific weights for features at specific locations.

Why MLP is not an optimal choice?

3. Lack of Translation Invariance

- MLPs are not robust to an object's position.
- The model learns specific weights for features at specific locations.

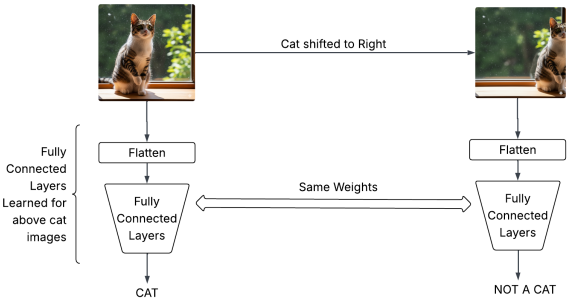


Figure: Illustration of Lack of Translation Invariance in Fully Connected Models

Convolutional Neural Networks

Efficiently modelling the Vision Tasks

From MLP to Convolution: The Starting Point

Treating Images as Spatially Structured Tensors

From MLP to Convolution: The Starting Point

Treating Images as Spatially Structured Tensors

Let's start by thinking of a Multi-Layer Perceptron (MLP) for images.

- We consider an image $\mathbf{X}$ and its hidden representation $\mathbf{H}$ as matrices of the same shape.
- Let $X_{i,j}$ be the pixel at location (i, j) and $H_{i,j}$ be the corresponding hidden unit.

From MLP to Convolution: The Starting Point

Treating Images as Spatially Structured Tensors

Let's start by thinking of a Multi-Layer Perceptron (MLP) for images.

- We consider an image $\mathbf{X}$ and its hidden representation $\mathbf{H}$ as matrices of the same shape.
- Let $X_{i,j}$ be the pixel at location (i, j) and $H_{i,j}$ be the corresponding hidden unit.

The Fully Connected Approach

A fully connected layer would require a 4th-order weight tensor $\mathbf{W}$ to connect every input pixel to every hidden unit:

$$H_{i,j} = \sum_{k,l} W_{i,j,k,l} X_{k,l}$$

Principle 1: Translation Invariance

Simplifying the Weight Tensor

We introduce a crucial inductive bias: **translation invariance**.

Principle 1: Translation Invariance

Simplifying the Weight Tensor

We introduce a crucial inductive bias: **translation invariance**.

- This means the pattern detector should work the same regardless of where it is in the image.

Principle 1: Translation Invariance

Simplifying the Weight Tensor

We introduce a crucial inductive bias: **translation invariance**.

- This means the pattern detector should work the same regardless of where it is in the image.
- A shift in the input $\mathbf{X}$ should lead to a corresponding shift in the hidden representation $\mathbf{H}$.

Principle 1: Translation Invariance

Simplifying the Weight Tensor

We introduce a crucial inductive bias: **translation invariance**.

- This means the pattern detector should work the same regardless of where it is in the image.
- A shift in the input $\mathbf{X}$ should lead to a corresponding shift in the hidden representation $\mathbf{H}$.

Applying the Constraint

This principle implies that the weights do not depend on the absolute pixel location (i, j), but only on the relative offset $(k - i, l - j)$.

Principle 1: Translation Invariance

Simplifying the Weight Tensor

We introduce a crucial inductive bias: **translation invariance**.

- This means the pattern detector should work the same regardless of where it is in the image.
- A shift in the input $\mathbf{X}$ should lead to a corresponding shift in the hidden representation $\mathbf{H}$.

Applying the Constraint

This principle implies that the weights do not depend on the absolute pixel location (i, j), but only on the relative offset $(k - i, l - j)$.

- We can re-index the weight tensor: $W_{i,j,k,l} \rightarrow V_{k-i,l-j}$.

Applying the Constraint

This principle implies that the weights do not depend on the absolute pixel location (i, j), but only on the relative offset $(k - i, l - j)$.

- We can re-index the weight tensor: $W_{i,j,k,l} \rightarrow V_{k-i,l-j}$.
- The layer's computation simplifies significantly:

$$H_{i,j} = \sum_{a,b} V_{a,b} X_{i+a,j+b}$$

Applying the Constraint

This principle implies that the weights do not depend on the absolute pixel location (i, j), but only on the relative offset $(k - i, l - j)$.

- We can re-index the weight tensor: $W_{i,j,k,l} \rightarrow V_{k-i,l-j}$.
- The layer's computation simplifies significantly:

$$H_{i,j} = \sum_{a,b} V_{a,b} X_{i+a,j+b}$$

This is a **convolution**! The number of parameters is no longer tied to the image size, only to the size of the filter $\mathbf{V}$.

Principle 2: Locality

Focusing on What's Nearby

We introduce a second inductive bias: **locality**.

Principle 2: Locality

Focusing on What's Nearby

We introduce a second inductive bias: **locality**.

- To understand what's happening at location (i, j), we only need to look at the pixels in its immediate vicinity.
- Information from pixels far away is less relevant.

Principle 2: Locality

Focusing on What's Nearby

We introduce a second inductive bias: **locality**.

- To understand what's happening at location (i, j), we only need to look at the pixels in its immediate vicinity.
- Information from pixels far away is less relevant.

Applying the Constraint

We enforce locality by setting the filter weights $V_{a,b}$ to zero outside of a small window, say $|a| > \Delta$ or $|b| > \Delta$. The computation becomes:

$$H_{i,j} = \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} V_{a,b} X_{i+a,j+b}$$

Applying the Constraint

We enforce locality by setting the filter weights $V_{a,b}$ to zero outside of a small window, say $|a| > \Delta$ or $|b| > \Delta$. The computation becomes:

$$H_{i,j} = \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} V_{a,b} X_{i+a,j+b}$$

The Convolutional Layer

This equation defines a **convolutional layer**. The learnable tensor **V** is called the **convolution kernel** or **filter**. The number of parameters is now tiny (e.g., a few hundred) and independent of the image size.

A Note on Terminology: Convolution

Mathematical Definition vs. Deep Learning Practice

Let's briefly review the formal mathematical definition of convolution for 2D tensors:

A Note on Terminology: Convolution

Mathematical Definition vs. Deep Learning Practice

Let's briefly review the formal mathematical definition of convolution for 2D tensors:

Mathematical Convolution

$$(\mathbf{V} * \mathbf{X})_{i,j} = \sum_{a,b} V_{a,b} X_{i-a,j-b}$$

Notice the subtraction of indices $(i - a, j - b)$. This involves "flipping" the kernel before sliding it over the input.

A Note on Terminology: Convolution

Mathematical Definition vs. Deep Learning Practice

Let's briefly review the formal mathematical definition of convolution for 2D tensors:

Mathematical Convolution

$$(\mathbf{V} * \mathbf{X})_{i,j} = \sum_{a,b} V_{a,b} X_{i-a,j-b}$$

Notice the subtraction of indices $(i - a, j - b)$. This involves "flipping" the kernel before sliding it over the input.

Deep Learning "Convolution"

$$H_{i,j} = \sum_{a,b} V_{a,b} X_{i+a,j+b}$$

This is technically a **cross-correlation**.

A Note on Terminology: Convolution

Mathematical Definition vs. Deep Learning Practice

The Difference in formulations

The term **convolution** in deep learning has a different mathematical definition than in pure mathematics. What we use is technically called **cross-correlation**.

A Note on Terminology: Convolution

Mathematical Definition vs. Deep Learning Practice

The Difference in formulations

The term **convolution** in deep learning has a different mathematical definition than in pure mathematics. What we use is technically called **cross-correlation**.

Does it matter?

A Note on Terminology: Convolution

Mathematical Definition vs. Deep Learning Practice

The Difference in formulations

The term **convolution** in deep learning has a different mathematical definition than in pure mathematics. What we use is technically called **cross-correlation**.

Does it matter? Not really. Since the kernel **V** is learned during training, the network can simply learn the flipped version of the kernel if needed. The terms are used interchangeably in deep learning.

Putting It All Together: Channels

Handling Multi-Dimensional Inputs and Outputs

Real-world images aren't 2D; they are 3rd-order tensors with color channels (e.g., Red, Green, Blue).

- Input $\mathbf{X}$ has shape: (height, width, **input channels**).

Putting It All Together: Channels

Handling Multi-Dimensional Inputs and Outputs

Real-world images aren't 2D; they are 3rd-order tensors with color channels (e.g., Red, Green, Blue).

- Input **X** has shape: (height, width, **input channels**).
- We also want our hidden representations **H** to have multiple channels, often called **feature maps**. Each map can learn to detect a different feature.

Putting It All Together: Channels

Handling Multi-Dimensional Inputs and Outputs

Real-world images aren't 2D; they are 3rd-order tensors with color channels (e.g., Red, Green, Blue).

- Input **X** has shape: (height, width, **input channels**).
- We also want our hidden representations **H** to have multiple channels, often called **feature maps**. Each map can learn to detect a different feature.
- Hidden Representation **H** has shape: (height, width, **output channels**).

Putting It All Together: Channels

Handling Multi-Dimensional Inputs and Outputs

Real-world images aren't 2D; they are 3rd-order tensors with color channels (e.g., Red, Green, Blue).

- Input **X** has shape: (height, width, **input channels**).
- We also want our hidden representations **H** to have multiple channels, often called **feature maps**. Each map can learn to detect a different feature.
- Hidden Representation **H** has shape: (height, width, **output channels**).

The General Convolutional Layer

Our kernel **W** becomes a 4th-order tensor, mapping from input channels to output channels. Let c_i be the input channel index and c_o be the output channel index. The full equation for a convolutional layer is:

$$H_{i,j,c_o} = \sum_{c_i} \sum_{a,b} W_{a,b,c_i,c_o} X_{i+a,j+b,c_i} + \text{bias}_{c_o}$$

The Convolution Layer in CNNs

The **convolution layer** is the core building block of a CNN. It performs the convolution operation.

The Convolution Layer in CNNs

The **convolution layer** is the core building block of a CNN. It performs the convolution operation.

Key Components:

- **Input:** A matrix of pixel values (e.g., a grayscale image). For a color image, this is a 3D tensor with dimensions (height, width, channels).

The Convolution Layer in CNNs

The **convolution layer** is the core building block of a CNN. It performs the convolution operation.

Key Components:

- **Input:** A matrix of pixel values (e.g., a grayscale image). For a color image, this is a 3D tensor with dimensions (height, width, channels).
- **Filter (or Kernel):** A small matrix of learnable weights. This filter slides or "convolves" across the input data.

The Convolution Layer in CNNs

The **convolution layer** is the core building block of a CNN. It performs the convolution operation.

Key Components:

- **Input:** A matrix of pixel values (e.g., a grayscale image). For a color image, this is a 3D tensor with dimensions (height, width, channels).
- **Filter (or Kernel):** A small matrix of learnable weights. This filter slides or "convolves" across the input data.
- **Feature Map (or Activation Map):** The output of the convolution. It represents the "activations" or responses of the filter at every spatial position of the input.

The Convolution Layer in CNNs

Key Note

The convolution operation involves an element-wise multiplication of the filter with a portion of the input, followed by a summation.

The Convolution Layer in CNNs

Key Note

The convolution operation involves an element-wise multiplication of the filter with a portion of the input, followed by a summation.

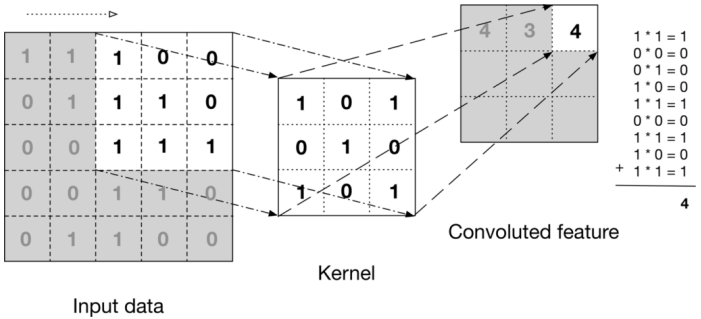


Figure: The 3x3 filter slides over the input image. At each position, it computes a dot product to generate a single value in the output feature map.

Edge Detection using Convolution Operation

Let's see a simple, intuitive example: detecting a horizontal edge.

Edge Detection using Convolution Operation

Let's see a simple, intuitive example: detecting a horizontal edge.

Input Image (6x6)

$$\begin{pmatrix} 30 & 30 & 30 & 30 & 30 & 30 \\ 30 & 30 & 30 & 30 & 30 & 30 \\ 30 & 30 & 30 & 30 & 30 & 30 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Edge Detection using Convolution Operation

Let's see a simple, intuitive example: detecting a horizontal edge.

Input Image (6x6) **Filter (3x3)**

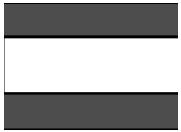
$$\begin{pmatrix} 30 & 30 & 30 & 30 & 30 & 30 \\ 30 & 30 & 30 & 30 & 30 & 30 \\ 30 & 30 & 30 & 30 & 30 & 30 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} * \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix}$$

Edge Detection using Convolution Operation

Let's see a simple, intuitive example: detecting a horizontal edge.

Input Image (6x6)

$$\begin{pmatrix} 30 & 30 & 30 & 30 & 30 & 30 \\ 30 & 30 & 30 & 30 & 30 & 30 \\ 30 & 30 & 30 & 30 & 30 & 30 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} * \quad \textbf{Filter (3x3)} \quad \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix} = \quad \textbf{Output Feature Map (4x4)} \quad \begin{pmatrix} 0 & 0 & 0 & 0 \\ 90 & 90 & 90 & 90 \\ 90 & 90 & 90 & 90 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$



Edge Detection using Convolution Operation

Observations

- The filter is designed to have a high response where there is a sharp change from a high value (30) on the top to a low value (0) on the bottom.

Edge Detection using Convolution Operation

Observations

- The filter is designed to have a high response where there is a sharp change from a high value (30) on the top to a low value (0) on the bottom.
- The output map has high values (90) exactly where the vertical edge was located in the input image.

Edge Detection using Convolution Operation

Observations

- The filter is designed to have a high response where there is a sharp change from a high value (30) on the top to a low value (0) on the bottom.
- The output map has high values (90) exactly where the vertical edge was located in the input image.

Key Fact

1. In the previous example, the filter used is a traditional edge detection filter called as *Prewitt Filter*

Edge Detection using Convolution Operation

Observations

- The filter is designed to have a high response where there is a sharp change from a high value (30) on the top to a low value (0) on the bottom.
- The output map has high values (90) exactly where the vertical edge was located in the input image.

Key Fact

1. In the previous example, the filter used is a traditional edge detection filter called as *Prewitt Filter*
2. **In a CNN, the filter/kernel matrices are learnt by the network.**

Learning a Convolutional Kernel

How does the Network learn?

1. The filter weights are initialized randomly: $W \sim \mathcal{N}(0, \sigma^2)$

Learning a Convolutional Kernel

How does the Network learn?

1. The filter weights are initialized randomly: $W \sim \mathcal{N}(0, \sigma^2)$
2. During training, the network makes a prediction.

$$\hat{y} = f_W(X)$$

Learning a Convolutional Kernel

How does the Network learn?

1. The filter weights are initialized randomly: $W \sim \mathcal{N}(0, \sigma^2)$
2. During training, the network makes a prediction.

$$\hat{y} = f_W(X)$$

3. The error (loss) between the prediction and the true label is calculated.

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Learning a Convolutional Kernel

How does a Network learn?

4. Using **backpropagation** and **gradient descent**, the filter weights are adjusted slightly to minimize this error.

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}$$

The above process is repeated for every image and for multiple iterations till the loss converges.

Learning a Convolutional Kernel

How does a Network learn?

4. Using **backpropagation** and **gradient descent**, the filter weights are adjusted slightly to minimize this error.

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}$$

The above process is repeated for every image and for multiple iterations till the loss converges.

Outcome

Over many iterations, the filters evolve to detect features (edges, colors, textures, patterns) that are most useful for the specific task (e.g., classifying cats vs. dogs).

Feature Maps

Key details about Feature Maps

- A feature map is the output of one convolutional filter applied to the input.

Feature Maps

Key details about Feature Maps

- A feature map is the output of one convolutional filter applied to the input.
- Since a convolutional layer typically has many filters, its full output is a 3D volume where each 2D slice is a feature map for a specific feature.

Feature Maps

Key details about Feature Maps

- A feature map is the output of one convolutional filter applied to the input.
- Since a convolutional layer typically has many filters, its full output is a 3D volume where each 2D slice is a feature map for a specific feature.
- For example, one map might activate for vertical edges, another for green patches, and a third for a specific texture.

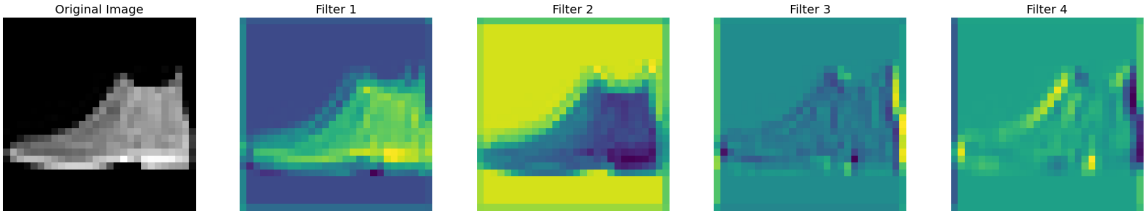


Figure: Output feature maps from different kernels for a same image

Receptive Field & Kernel Size

Key details

- The receptive field of a neuron is the size of the region in the **original input image** that affects its activation.

Receptive Field & Kernel Size

Key details

- The receptive field of a neuron is the size of the region in the **original input image** that affects its activation.
- In the first conv layer, the receptive field is just the filter size (e.g., 3×3).

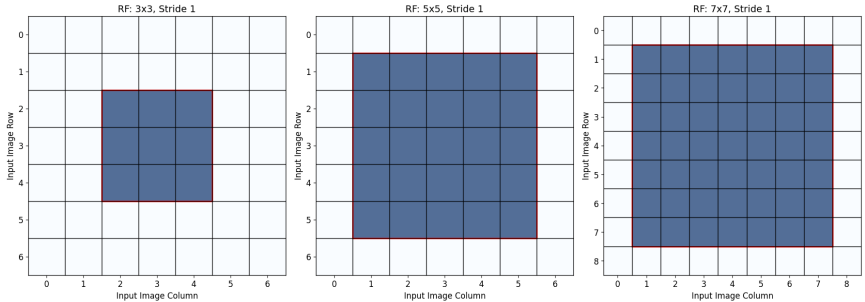


Figure: Receptive field varies with kernel size

Receptive Field & Layers

Key details

- As we stack more convolutional layers, the receptive field size **grows**. A neuron in Layer 2 is looking at neurons from Layer 1, which in turn look at the input image.

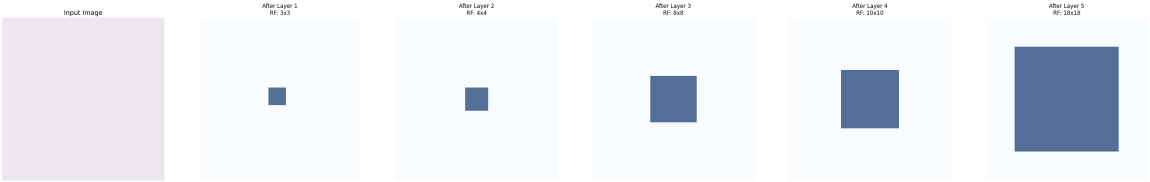


Figure: Receptive field grows as we stack more convolutional layers

Hierarchal Learning

Key details about Receptive Field

- This allows the network to build a **hierarchy of features**:
  - *Layer 1* learns simple edges from pixels.
  - *Layer 2* learns corners and contours from edges.
  - *Layer 3* learns parts of objects (like an eye or a nose) from corners.
  - *Deeper layers* learn to recognize entire objects.

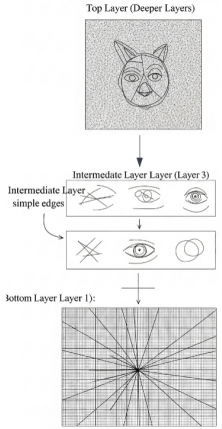


Figure: Image features learnt by each layer (source: Imagen)

Hyperparameters of Convolutional Neural Network

Understanding Kernel size, Stride and Padding

CNN Hyperparameters: Kernel Size

Definition

- The **kernel size** (or filter size) defines the dimensions (height $\times$ width) of the convolution window that slides over the input data.

CNN Hyperparameters: Kernel Size

Definition

- The **kernel size** (or filter size) defines the dimensions (height $\times$ width) of the convolution window that slides over the input data.
- It determines the size of the **receptive field**—the area of the input that the filter considers to compute a single value in the output feature map.

CNN Hyperparameters: Kernel Size

Definition

- The **kernel size** (or filter size) defines the dimensions (height $\times$ width) of the convolution window that slides over the input data.
- It determines the size of the **receptive field**—the area of the input that the filter considers to compute a single value in the output feature map.

Common Choices

- **3×3**: The most widely used size. Stacking multiple 3×3 layers can capture a larger receptive field with fewer parameters and more non-linearity than one large kernel.
- **5×5, 7×7**: Used to capture larger spatial patterns, often in the initial layers of a network.

CNN Hyperparameters: Kernel Size

Definition

- The **kernel size** (or filter size) defines the dimensions (height $\times$ width) of the convolution window that slides over the input data.
- It determines the size of the **receptive field**—the area of the input that the filter considers to compute a single value in the output feature map.

Common Choices

- **3×3**: The most widely used size. Stacking multiple 3×3 layers can capture a larger receptive field with fewer parameters and more non-linearity than one large kernel.
- **5×5, 7×7**: Used to capture larger spatial patterns, often in the initial layers of a network.

Trade-off

Smaller kernels learn local, fine-grained features, while larger kernels capture more global, coarse features at a higher computational cost.

CNN Kernel Visualization

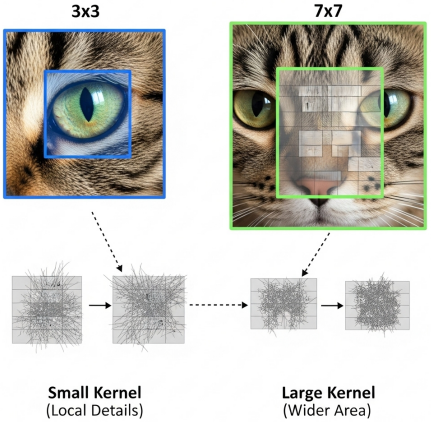


Figure: A smaller kernel has a smaller receptive field, focusing on local details. A larger kernel captures information from a wider area. (source: Imagen)

Importance of 1x1 Convolutions

What is 1×1 convolution?

A convolution with a 1×1 kernel might seem strange—it only looks at one pixel at a time! However, it is a very powerful technique, also known as a "Network in Network" layer.

Importance of 1x1 Convolutions

What is 1×1 convolution?

A convolution with a 1×1 kernel might seem strange—it only looks at one pixel at a time! However, it is a very powerful technique, also known as a "Network in Network" layer.

Use Case 1: Dimensionality Reduction

- Imagine an input volume of $56 \times 56 \times 192$.

Importance of 1x1 Convolutions

What is 1×1 convolution?

A convolution with a 1×1 kernel might seem strange—it only looks at one pixel at a time! However, it is a very powerful technique, also known as a "Network in Network" layer.

Use Case 1: Dimensionality Reduction

- Imagine an input volume of $56 \times 56 \times 192$.
- Applying 32 filters of size $1 \times 1 \times 192$ results in an output volume of $56 \times 56 \times 32$.

Importance of 1x1 Convolutions

What is 1×1 convolution?

A convolution with a 1×1 kernel might seem strange—it only looks at one pixel at a time! However, it is a very powerful technique, also known as a "Network in Network" layer.

Use Case 1: Dimensionality Reduction

- Imagine an input volume of $56 \times 56 \times 192$.
- Applying 32 filters of size $1 \times 1 \times 192$ results in an output volume of $56 \times 56 \times 32$.
- We have reduced the number of channels (the depth) from 192 to 32 without changing the spatial dimensions (height and width).

Importance of 1x1 Convolutions

What is 1×1 convolution?

A convolution with a 1×1 kernel might seem strange—it only looks at one pixel at a time! However, it is a very powerful technique, also known as a "Network in Network" layer.

Use Case 1: Dimensionality Reduction

- Imagine an input volume of $56 \times 56 \times 192$.
- Applying 32 filters of size $1 \times 1 \times 192$ results in an output volume of $56 \times 56 \times 32$.
- We have reduced the number of channels (the depth) from 192 to 32 without changing the spatial dimensions (height and width).

Use Case 2: Adding Non-linearity

- A 1×1 convolution is essentially a fully connected layer operating on the channel dimension for each pixel.

Importance of 1x1 Convolutions

What is 1×1 convolution?

A convolution with a 1×1 kernel might seem strange—it only looks at one pixel at a time! However, it is a very powerful technique, also known as a "Network in Network" layer.

Use Case 1: Dimensionality Reduction

- Imagine an input volume of $56 \times 56 \times 192$.
- Applying 32 filters of size $1 \times 1 \times 192$ results in an output volume of $56 \times 56 \times 32$.
- We have reduced the number of channels (the depth) from 192 to 32 without changing the spatial dimensions (height and width).

Use Case 2: Adding Non-linearity

- A 1×1 convolution is essentially a fully connected layer operating on the channel dimension for each pixel.
- By applying a non-linear activation function (like ReLU) after the 1×1 convolution, the network can learn more complex relationships between channels.

Importance of 1x1 Convolutions

What is 1×1 convolution?

A convolution with a 1×1 kernel might seem strange—it only looks at one pixel at a time! However, it is a very powerful technique, also known as a "Network in Network" layer.

Use Case 1: Dimensionality Reduction

- Imagine an input volume of $56 \times 56 \times 192$.
- Applying 32 filters of size $1 \times 1 \times 192$ results in an output volume of $56 \times 56 \times 32$.
- We have reduced the number of channels (the depth) from 192 to 32 without changing the spatial dimensions (height and width).

Use Case 2: Adding Non-linearity

- A 1×1 convolution is essentially a fully connected layer operating on the channel dimension for each pixel.
- By applying a non-linear activation function (like ReLU) after the 1×1 convolution, the network can learn more complex relationships between channels.
- This increases the model's expressive power without affecting its receptive field.

CNN Hyperparameters: Stride

Definition

- **Stride** defines the step size, or the number of pixels the convolutional kernel moves over the input feature map at a time.

CNN Hyperparameters: Stride

Definition

- **Stride** defines the step size, or the number of pixels the convolutional kernel moves over the input feature map at a time.
- A stride of 1 ($S = 1$) means the kernel shifts one pixel at a time (horizontally or vertically). This is the most common setting.

CNN Hyperparameters: Stride

Definition

- **Stride** defines the step size, or the number of pixels the convolutional kernel moves over the input feature map at a time.
- A stride of 1 ($S = 1$) means the kernel shifts one pixel at a time (horizontally or vertically). This is the most common setting.

Primary Effect: Downsampling

- Using a stride greater than 1 reduces the spatial dimensions (height and width) of the output feature map.
- This is often used in place of pooling layers to decrease the computational load and help the network learn more abstract representations by reducing spatial resolution.

Strided Convolution Illustration

Input Feature Map (5×5)

$x_{0,0}$	$x_{1,0}$	$x_{2,0}$	$x_{3,0}$	$x_{4,0}$
$x_{0,1}$	$x_{1,1}$	$x_{2,1}$	$x_{3,1}$	$x_{4,1}$
$x_{0,2}$	$x_{1,2}$	$x_{2,2}$	$x_{3,2}$	$x_{4,2}$
$x_{0,3}$	$x_{1,3}$	$x_{2,3}$	$x_{3,3}$	$x_{4,3}$
$x_{0,4}$	$x_{1,4}$	$x_{2,4}$	$x_{3,4}$	$x_{4,4}$

3×3 Kernel, Stride = 2

$w_{0,0}$	$w_{1,0}$	$w_{2,0}$
$w_{0,1}$	$w_{1,1}$	$w_{2,1}$
$w_{0,2}$	$w_{1,2}$	$w_{2,2}$

$\Rightarrow$

Output Feature Map (2×2)

$y_{0,0}$	$y_{0,1}$
$y_{1,0}$	$y_{1,1}$

Instructions:

1. Position kernel at (0,0) $\rightarrow$ compute $y_{0,0}$

Strided Convolution Illustration

Input Feature Map (5×5)

$x_{0,0}$	$x_{1,0}$	$x_{2,0}$	$x_{3,0}$	$x_{4,0}$
$x_{0,1}$	$x_{1,1}$	$x_{2,1}$	$x_{3,1}$	$x_{4,1}$
$x_{0,2}$	$x_{1,2}$	$x_{2,2}$	$x_{3,2}$	$x_{4,2}$
$x_{0,3}$	$x_{1,3}$	$x_{2,3}$	$x_{3,3}$	$x_{4,3}$
$x_{0,4}$	$x_{1,4}$	$x_{2,4}$	$x_{3,4}$	$x_{4,4}$

$\Rightarrow$

Output Feature Map (2×2)

$y_{0,0}$	$y_{0,1}$
$y_{1,0}$	$y_{1,1}$

3×3 Kernel, Stride = 2

$w_{0,0}$	$w_{1,0}$	$w_{2,0}$
$w_{0,1}$	$w_{1,1}$	$w_{2,1}$
$w_{0,2}$	$w_{1,2}$	$w_{2,2}$

Instructions:

2. Move kernel by stride=2 to (0,2) $\rightarrow$ compute $y_{0,1}$

Strided Convolution Illustration

Input Feature Map (5×5)

$x_{0,0}$	$x_{1,0}$	$x_{2,0}$	$x_{3,0}$	$x_{4,0}$
$x_{0,1}$	$x_{1,1}$	$x_{2,1}$	$x_{3,1}$	$x_{4,1}$
$x_{0,2}$	$x_{1,2}$	$x_{2,2}$	$x_{3,2}$	$x_{4,2}$
$x_{0,3}$	$x_{1,3}$	$x_{2,3}$	$x_{3,3}$	$x_{4,3}$
$x_{0,4}$	$x_{1,4}$	$x_{2,4}$	$x_{3,4}$	$x_{4,4}$

$\Rightarrow$

Output Feature Map (2×2)

$y_{0,0}$	$y_{0,1}$
$y_{1,0}$	$y_{1,1}$

Instructions:

3×3 Kernel, Stride = 2

$w_{0,0}$	$w_{1,0}$	$w_{2,0}$
$w_{0,1}$	$w_{1,1}$	$w_{2,1}$
$w_{0,2}$	$w_{1,2}$	$w_{2,2}$

3. Move kernel to (2,0) $\rightarrow$ compute $y_{1,0}$

Strided Convolution Illustration

Input Feature Map (5×5)

$x_{0,0}$	$x_{1,0}$	$x_{2,0}$	$x_{3,0}$	$x_{4,0}$
$x_{0,1}$	$x_{1,1}$	$x_{2,1}$	$x_{3,1}$	$x_{4,1}$
$x_{0,2}$	$x_{1,2}$	$x_{2,2}$	$x_{3,2}$	$x_{4,2}$
$x_{0,3}$	$x_{1,3}$	$x_{2,3}$	$x_{3,3}$	$x_{4,3}$
$x_{0,4}$	$x_{1,4}$	$x_{2,4}$	$x_{3,4}$	$x_{4,4}$

$\Rightarrow$

Output Feature Map (2×2)

$y_{0,0}$	$y_{0,1}$
$y_{1,0}$	$y_{1,1}$

Instructions:

3×3 Kernel, Stride = 2

$w_{0,0}$	$w_{1,0}$	$w_{2,0}$
$w_{0,1}$	$w_{1,1}$	$w_{2,1}$
$w_{0,2}$	$w_{1,2}$	$w_{2,2}$

4. Move kernel to (2,2) $\rightarrow$ compute $y_{1,1}$

Stride and Output Size Calculation

For a convolution operation:

Output Size Formula

$$\text{Output Size} = \left\lfloor \frac{\text{Input Size} - \text{Kernel Size}}{\text{Stride}} \right\rfloor + 1$$

Stride and Output Size Calculation

For a convolution operation:

Output Size Formula

$$\text{Output Size} = \left\lfloor \frac{\text{Input Size} - \text{Kernel Size}}{\text{Stride}} \right\rfloor + 1$$

Examples

- Stride 1: $\left\lfloor \frac{4-3}{1} \right\rfloor + 1 = 2$

Stride and Output Size Calculation

For a convolution operation:

Output Size Formula

$$\text{Output Size} = \left\lfloor \frac{\text{Input Size} - \text{Kernel Size}}{\text{Stride}} \right\rfloor + 1$$

Examples

- Stride 1: $\left\lfloor \frac{4-3}{1} \right\rfloor + 1 = 2$
- Stride 2: $\left\lfloor \frac{5-3}{2} \right\rfloor + 1 = 2$

Note: This assumes no padding

CNN Hyperparameters: Padding

Definition

Padding is the process of adding extra pixels (usually with a value of 0, called *zero-padding*) around the border of an input feature map.

CNN Hyperparameters: Padding

Definition

Padding is the process of adding extra pixels (usually with a value of 0, called *zero-padding*) around the border of an input feature map.

Why use Padding?

1. **To control the output size.** Without padding, the spatial dimensions shrink with each convolution. Padding allows us to preserve the input dimensions.

CNN Hyperparameters: Padding

Definition

Padding is the process of adding extra pixels (usually with a value of 0, called *zero-padding*) around the border of an input feature map.

Why use Padding?

1. **To control the output size.** Without padding, the spatial dimensions shrink with each convolution. Padding allows us to preserve the input dimensions.
   - **"Same" Padding:** Adds enough padding to make the output feature map have the same height and width as the input.

CNN Hyperparameters: Padding

Definition

Padding is the process of adding extra pixels (usually with a value of 0, called *zero-padding*) around the border of an input feature map.

Why use Padding?

1. **To control the output size.** Without padding, the spatial dimensions shrink with each convolution. Padding allows us to preserve the input dimensions.
   - **"Same" Padding:** Adds enough padding to make the output feature map have the same height and width as the input.
   - **"Valid" Padding:** No padding is applied. The output is smaller than the input.

CNN Hyperparameters: Padding

Definition

Padding is the process of adding extra pixels (usually with a value of 0, called *zero-padding*) around the border of an input feature map.

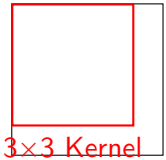
Why use Padding?

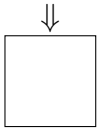
1. **To control the output size.** Without padding, the spatial dimensions shrink with each convolution. Padding allows us to preserve the input dimensions.
   - **"Same" Padding:** Adds enough padding to make the output feature map have the same height and width as the input.
   - **"Valid" Padding:** No padding is applied. The output is smaller than the input.
2. **To improve performance at the borders.** Pixels at the edges of an image are seen by the kernel fewer times than central pixels. Padding ensures that edge and corner pixels are given more consideration during convolution.

Padding in CNN Illustration

Without Padding ("Valid")

Input: 4×4





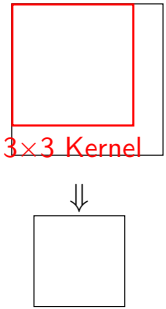
(Output: 2×2)

(Output shrinks)

Padding in CNN Illustration

Without Padding ("Valid")

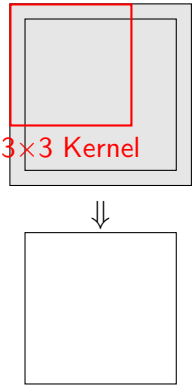
Input: 4×4



(Output: 2×2)
(Output shrinks)

With Zero-Padding ("Same")

Input: 4×4 Padded Input: 6×6



(Output: 4×4)
(Output size is preserved)

Convolution Layer Output

The size of the output feature map is controlled by three key hyperparameters:

Convolution Layer Output

The size of the output feature map is controlled by three key hyperparameters:

- **Kernel Size (K):** The dimensions of the filter (e.g., 3×3, 5×5).

Convolution Layer Output

The size of the output feature map is controlled by three key hyperparameters:

- **Kernel Size (K):** The dimensions of the filter (e.g., 3×3, 5×5).
- **Stride (S):** The number of pixels the filter moves at each step. A stride of 1 moves one pixel at a time, while a stride of 2 skips a pixel.

Convolution Layer Output

The size of the output feature map is controlled by three key hyperparameters:

- **Kernel Size (K):** The dimensions of the filter (e.g., 3×3, 5×5).
- **Stride (S):** The number of pixels the filter moves at each step. A stride of 1 moves one pixel at a time, while a stride of 2 skips a pixel.
- **Padding (P):** Adding zeros around the border of the input image. This allows us to control the output size and helps keep information at the borders.

Convolution Layer Output

The size of the output feature map is controlled by three key hyperparameters:

- **Kernel Size (K):** The dimensions of the filter (e.g., 3×3, 5×5).
- **Stride (S):** The number of pixels the filter moves at each step. A stride of 1 moves one pixel at a time, while a stride of 2 skips a pixel.
- **Padding (P):** Adding zeros around the border of the input image. This allows us to control the output size and helps keep information at the borders.

Output Size Formula

For an input of width W and height H, the output dimensions $(W_{out}, H_{out}, C_{out})$ are:

$$W_{out} = \frac{W - K + 2P}{S} + 1$$

$$H_{out} = \frac{H - K + 2P}{S} + 1$$

$$C_{out} = \text{Number of Convolution Filters}$$

Convolution Layer Output

Example 1: Basic Convolution

- **Input (W)**: 32×32
- **Kernel (K)**: 5×5
- **Padding (P)**: 0 (Valid)
- **Stride (S)**: 1
- **Number of Filters**: 5

$$O = \lfloor \frac{32-5+2(0)}{1} \rfloor + 1$$
$$O = \lfloor 27 \rfloor + 1 = 28$$

Output Size: $28 \times 28 \times 5$

Convolution Layer Output

Example 1: Basic Convolution

- **Input (W)**: 32×32
- **Kernel (K)**: 5×5
- **Padding (P)**: 0 (Valid)
- **Stride (S)**: 1
- **Number of Filters**: 5

$O = \lfloor \frac{32-5+2(0)}{1} \rfloor + 1$
$O = \lfloor 27 \rfloor + 1 = 28$

Output Size: $28 \times 28 \times 5$

Example 2: Downsampling

- **Input (W)**: 32×32
- **Kernel (K)**: 3×3
- **Padding (P)**: 1 (Same)
- **Stride (S)**: 2
- **Number of Filters**: 64

$O = \lfloor \frac{32-3+2(1)}{2} \rfloor + 1$
$O = \lfloor \frac{31}{2} \rfloor + 1 = 15 + 1 = 16$

Output Size: $16 \times 16 \times 64$

Other Components of Convolutional Neural Network

Pooling, BatchNorm and Dropout

Pooling Layer

Definition

A pooling layer performs non-linear **downsampling** to reduce the spatial dimensions (width and height) of the feature maps.

Pooling Layer

Definition

A pooling layer performs non-linear **downsampling** to reduce the spatial dimensions (width and height) of the feature maps.

Key Purposes

- **Reduce Computational Cost:** Decreases the number of parameters and computations in the network by making feature maps smaller.

Pooling Layer

Definition

A pooling layer performs non-linear **downsampling** to reduce the spatial dimensions (width and height) of the feature maps.

Key Purposes

- **Reduce Computational Cost:** Decreases the number of parameters and computations in the network by making feature maps smaller.
- **Increase Receptive Field:** Allows subsequent convolutional layers to see a larger area of the original input.

Pooling Layer

Definition

A pooling layer performs non-linear **downsampling** to reduce the spatial dimensions (width and height) of the feature maps.

Key Purposes

- **Reduce Computational Cost:** Decreases the number of parameters and computations in the network by making feature maps smaller.
- **Increase Receptive Field:** Allows subsequent convolutional layers to see a larger area of the original input.
- **Provide Translational Invariance:** Makes the network more robust to small shifts and distortions in the input image. The output of the pooling operation can remain the same even if the input features move slightly.

Types of Pooling Layers

Max Pooling

- Selects the **maximum** value from each patch.
- Most common type.
- Effective at capturing the most prominent features.

Average Pooling

- Computes the **average** value from each patch.
- Smooths the feature representation.

Illustration of Pooling Layers

Applying a 2×2 pooling operation with a stride of 2.

Max Pooling

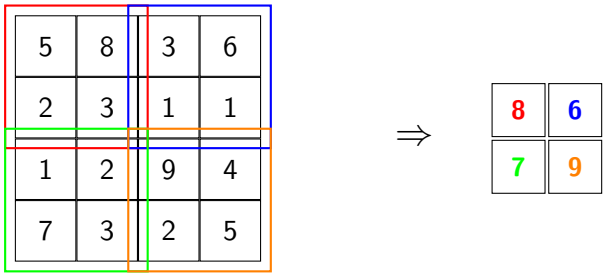
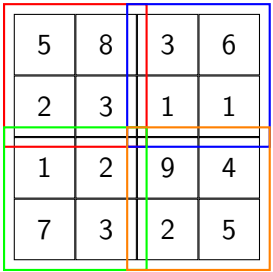


Figure: Takes the maximum value from each patch.

Illustration of Pooling Layers

Applying a 2×2 pooling operation with a stride of 2.

Average Pooling



$\Rightarrow$

4.50	2.75
3.25	5.00

Takes the average value of each patch.

Illustration of Pooling Layers

Applying a 2×2 pooling operation with a stride of 2.

Min Pooling

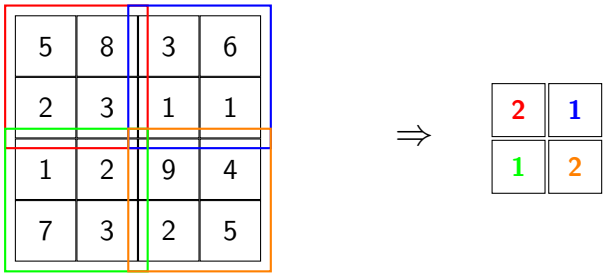


Figure: Takes the minimum value of each patch

Batch Normalization

The Problem: Internal Covariate Shift

During training, the distribution of each layer's inputs changes as the parameters of the preceding layers are updated. This slows down training and makes the network highly sensitive to weight initialization.

Batch Normalization

The Problem: Internal Covariate Shift

During training, the distribution of each layer's inputs changes as the parameters of the preceding layers are updated. This slows down training and makes the network highly sensitive to weight initialization.

Batch Normalization is a technique that standardizes the inputs to a layer for each mini-batch. It stabilizes the learning process and dramatically speeds up training.

Batch Normalization

The Problem: Internal Covariate Shift

During training, the distribution of each layer's inputs changes as the parameters of the preceding layers are updated. This slows down training and makes the network highly sensitive to weight initialization.

Batch Normalization is a technique that standardizes the inputs to a layer for each mini-batch. It stabilizes the learning process and dramatically speeds up training.

Key Benefits

- Allows for much **higher learning rates**, leading to faster convergence.
- **Reduces the need for careful initialization** of network weights.
- Acts as a form of **regularization**, sometimes reducing the need for Dropout.

Batch Normalization: Walkthrough

For a mini-batch of activations $B = \{x_1, \ldots, x_m\}$:

1. **Calculate Batch Mean:** Find the average of the mini-batch.

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

Batch Normalization: Walkthrough

For a mini-batch of activations $B = \{x_1, \ldots, x_m\}$:

1. **Calculate Batch Mean:** Find the average of the mini-batch.

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

2. **Calculate Batch Variance:** Find the variance of the mini-batch.

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2$$

Batch Normalization: Walkthrough

3. **Normalize:** Normalize each activation using the mean and variance.

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

(ϵ is a small constant for numerical stability)

Batch Normalization: Walkthrough

3. **Normalize:** Normalize each activation using the mean and variance.

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

(ϵ is a small constant for numerical stability)

4. **Scale and Shift:** The network learns two new parameters, γ (scale) and β (shift), to restore representative power.

$$y_i = \gamma \hat{x}_i + \beta$$

Batch Normalization: CNN Example Setup

Scenario: Consider a convolutional layer output with:

- Mini-batch size: $m = 4$ images
- Feature map size: 2×2 (simplified)
- Number of channels: $C = 1$ (single channel)

Activations from 4 images at a specific spatial location (i,j):

$$B = \{x_1 = 3.2, x_2 = 1.8, x_3 = 4.1, x_4 = 2.9\}$$

Note: In practice, BN is applied across all spatial locations for each channel, but we'll focus on one location for clarity.

Step 1: Calculate Batch Mean

Given: $B = \{3.2, 1.8, 4.1, 2.9\}$, $m = 4$

Formula:

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

Calculation:

$$\mu_B = \frac{1}{4}(3.2 + 1.8 + 4.1 + 2.9) \tag{1}$$

$$= \frac{1}{4}(12.0) \tag{2}$$

$$= 3.0 \tag{3}$$

Step 1: Calculate Batch Mean

Given: $B = \{3.2, 1.8, 4.1, 2.9\}$, $m = 4$

Formula:

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

Calculation:

$$\mu_B = \frac{1}{4}(3.2 + 1.8 + 4.1 + 2.9) \tag{1}$$

$$= \frac{1}{4}(12.0) \tag{2}$$

$$= 3.0 \tag{3}$$

Result: $\mu_B = 3.0$

Step 2: Calculate Batch Variance

Formula:

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

Calculation:

$$\sigma_B^2 = \frac{1}{4}[(3.2 - 3.0)^2 + (1.8 - 3.0)^2 + (4.1 - 3.0)^2 + (2.9 - 3.0)^2] \quad (4)$$

$$= \frac{1}{4}[(0.2)^2 + (-1.2)^2 + (1.1)^2 + (-0.1)^2] \quad (5)$$

$$= \frac{1}{4}[0.04 + 1.44 + 1.21 + 0.01] \quad (6)$$

$$= \frac{1}{4}(2.70) \quad (7)$$

$$= 0.675 \quad (8)$$

Step 2: Calculate Batch Variance

Formula:

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

Calculation:

$$\sigma_B^2 = \frac{1}{4}[(3.2 - 3.0)^2 + (1.8 - 3.0)^2 + (4.1 - 3.0)^2 + (2.9 - 3.0)^2] \quad (4)$$

$$= \frac{1}{4}[(0.2)^2 + (-1.2)^2 + (1.1)^2 + (-0.1)^2] \quad (5)$$

$$= \frac{1}{4}[0.04 + 1.44 + 1.21 + 0.01] \quad (6)$$

$$= \frac{1}{4}(2.70) \quad (7)$$

$$= 0.675 \quad (8)$$

Result: $\sigma_B^2 = 0.675$

Step 3: Normalize

Formula:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Given: $\mu_B = 3.0$, $\sigma_B^2 = 0.675$, $\epsilon = 1 \times 10^{-8}$ (for numerical stability)

Standard deviation: $\sqrt{\sigma_B^2 + \epsilon} = \sqrt{0.675} \approx 0.822$

Normalized values:

$$\hat{x}_1 = \frac{3.2 - 3.0}{0.822} = \frac{0.2}{0.822} \approx 0.243 \tag{9}$$

$$\hat{x}_2 = \frac{1.8 - 3.0}{0.822} = \frac{-1.2}{0.822} \approx -1.460 \tag{10}$$

$$\hat{x}_3 = \frac{4.1 - 3.0}{0.822} = \frac{1.1}{0.822} \approx 1.338 \tag{11}$$

$$\hat{x}_4 = \frac{2.9 - 3.0}{0.822} = \frac{-0.1}{0.822} \approx -0.122 \tag{12}$$

Step 4: Scale and Shift

Formula:

$$y_i = \gamma \hat{x}_i + \beta$$

Learnable parameters: $\gamma = 2.0$ (scale), $\beta = 0.5$ (shift)

Final outputs:

$$y_1 = 2.0 \times 0.243 + 0.5 = 0.486 + 0.5 = 0.986 \quad (13)$$

$$y_2 = 2.0 \times (-1.460) + 0.5 = -2.920 + 0.5 = -2.420 \quad (14)$$

$$y_3 = 2.0 \times 1.338 + 0.5 = 2.676 + 0.5 = 3.176 \quad (15)$$

$$y_4 = 2.0 \times (-0.122) + 0.5 = -0.244 + 0.5 = 0.256 \quad (16)$$

Step 4: Scale and Shift

Formula:

$$y_i = \gamma \hat{x}_i + \beta$$

Learnable parameters: $\gamma = 2.0$ (scale), $\beta = 0.5$ (shift)

Final outputs:

$$y_1 = 2.0 \times 0.243 + 0.5 = 0.486 + 0.5 = 0.986 \quad (13)$$

$$y_2 = 2.0 \times (-1.460) + 0.5 = -2.920 + 0.5 = -2.420 \quad (14)$$

$$y_3 = 2.0 \times 1.338 + 0.5 = 2.676 + 0.5 = 3.176 \quad (15)$$

$$y_4 = 2.0 \times (-0.122) + 0.5 = -0.244 + 0.5 = 0.256 \quad (16)$$

Summary:

- Original: $\{3.2, 1.8, 4.1, 2.9\}$
- After BN: $\{0.986, -2.420, 3.176, 0.256\}$

Batch Normalization in CNNs: Key Points

In practice for CNNs:

- BN is applied **per channel** across the batch and spatial dimensions
- For a feature map of shape (N, C, H, W):
  - Calculate μ and σ^2 across (N, H, W) dimensions
  - Each channel has its own γ and β parameters

Batch Normalization in CNNs: Key Points

In practice for CNNs:

- BN is applied **per channel** across the batch and spatial dimensions
- For a feature map of shape (N, C, H, W):
  - Calculate μ and σ^2 across (N, H, W) dimensions
  - Each channel has its own γ and β parameters
- **Training vs. Inference:**
  - Training: Use batch statistics (μ_B, σ_B^2)
  - Inference: Use running averages of training statistics

Running Averages in Batch Normalization: The Problem

Why do we need running averages?

Training vs. Inference Dilemma:

- **Training:** We have mini-batches (e.g., 32 images)
  - Can compute reliable μ_B and σ_B^2 from the batch

Running Averages in Batch Normalization: The Problem

Why do we need running averages?

Training vs. Inference Dilemma:

- **Training:** We have mini-batches (e.g., 32 images)
  - Can compute reliable μ_B and σ_B^2 from the batch
- **Inference:** We might have only 1 image (or small batches)
  - Computing μ_B and σ_B^2 from 1 sample is meaningless!
  - Single image: $\mu_B = x_1$, $\sigma_B^2 = 0$. **This is the Problem!**

Running Averages in Batch Normalization: The Problem

Why do we need running averages?

Training vs. Inference Dilemma:

- **Training:** We have mini-batches (e.g., 32 images)
  - Can compute reliable μ_B and σ_B^2 from the batch
- **Inference:** We might have only 1 image (or small batches)
  - Computing μ_B and σ_B^2 from 1 sample is meaningless!
  - Single image: $\mu_B = x_1$, $\sigma_B^2 = 0$. **This is the Problem!**
- **Solution:** Use population statistics estimated during training

Running Averages in Batch Normalization: The Problem

Why do we need running averages?

Training vs. Inference Dilemma:

- **Training:** We have mini-batches (e.g., 32 images)
  - Can compute reliable μ_B and σ_B^2 from the batch
- **Inference:** We might have only 1 image (or small batches)
  - Computing μ_B and σ_B^2 from 1 sample is meaningless!
  - Single image: $\mu_B = x_1$, $\sigma_B^2 = 0$. **This is the Problem!**
- **Solution:** Use population statistics estimated during training
- **Framework behavior:**
  - PyTorch: `model.train()` vs `model.eval()`
  - TensorFlow: `training=True/False` parameter

Running Averages in Batch Normalization: The Problem

Why do we need running averages?

Training vs. Inference Dilemma:

- **Training:** We have mini-batches (e.g., 32 images)
  - Can compute reliable μ_B and σ_B^2 from the batch
- **Inference:** We might have only 1 image (or small batches)
  - Computing μ_B and σ_B^2 from 1 sample is meaningless!
  - Single image: $\mu_B = x_1$, $\sigma_B^2 = 0$. **This is the Problem!**
- **Solution:** Use population statistics estimated during training
- **Framework behavior:**
  - PyTorch: `model.train()` vs `model.eval()`
  - TensorFlow: `training=True/False` parameter

Key Insight: We need stable, batch-size-independent statistics for inference

Running Averages: How They Work

During Training - Dual Process:

For each mini-batch, we do **TWO** things:

1. **Forward Pass:** Use current batch statistics

$$\text{BN Output: } y_i = \gamma \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

Running Averages: How They Work

During Training - Dual Process:

For each mini-batch, we do **TWO** things:

1. **Forward Pass:** Use current batch statistics

$$\text{BN Output: } y_i = \gamma \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

2. **Update Running Statistics:** Exponential Moving Average (EMA)

$$\mu_{\text{running}} \leftarrow (1 - \alpha)\mu_{\text{running}} + \alpha \mu_B \tag{17}$$

$$\sigma^2_{\text{running}} \leftarrow (1 - \alpha)\sigma^2_{\text{running}} + \alpha \sigma_B^2 \tag{18}$$

where α is the momentum (typically 0.1)

Running Averages: How They Work

During Training - Dual Process:

For each mini-batch, we do **TWO** things:

1. **Forward Pass:** Use current batch statistics

$$\text{BN Output: } y_i = \gamma \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

2. **Update Running Statistics:** Exponential Moving Average (EMA)

$$\mu_{\text{running}} \leftarrow (1 - \alpha)\mu_{\text{running}} + \alpha \mu_B \tag{17}$$

$$\sigma^2_{\text{running}} \leftarrow (1 - \alpha)\sigma^2_{\text{running}} + \alpha \sigma_B^2 \tag{18}$$

where α is the momentum (typically 0.1)

Note: Running statistics are not used during training forward pass, only updated!

Running Averages: Numerical Example

Scenario: Training with momentum $\alpha = 0.1$

Initial values: $\mu_{\text{running}} = 0$, $\sigma^2_{\text{running}} = 1$

Batch 1: $\mu_{B1} = 2.0$, $\sigma^2_{B1} = 0.5$

$$\mu_{\text{running}} = (1 - 0.1) \times 0 + 0.1 \times 2.0 = 0.2 \tag{19}$$

$$\sigma^2_{\text{running}} = (1 - 0.1) \times 1 + 0.1 \times 0.5 = 0.95 \tag{20}$$

Running Averages: Numerical Example

Scenario: Training with momentum $\alpha = 0.1$

Initial values: $\mu_{\text{running}} = 0$, $\sigma^2_{\text{running}} = 1$

Batch 1: $\mu_{B1} = 2.0$, $\sigma^2_{B1} = 0.5$

$$\mu_{\text{running}} = (1 - 0.1) \times 0 + 0.1 \times 2.0 = 0.2 \tag{19}$$

$$\sigma^2_{\text{running}} = (1 - 0.1) \times 1 + 0.1 \times 0.5 = 0.95 \tag{20}$$

Batch 2: $\mu_{B2} = 3.0$, $\sigma^2_{B2} = 0.8$

$$\mu_{\text{running}} = 0.9 \times 0.2 + 0.1 \times 3.0 = 0.48 \tag{21}$$

$$\sigma^2_{\text{running}} = 0.9 \times 0.95 + 0.1 \times 0.8 = 0.935 \tag{22}$$

Running Averages: Numerical Example

Scenario: Training with momentum $\alpha = 0.1$

Initial values: $\mu_{\text{running}} = 0$, $\sigma^2_{\text{running}} = 1$

Batch 1: $\mu_{B1} = 2.0$, $\sigma^2_{B1} = 0.5$

$$\mu_{\text{running}} = (1 - 0.1) \times 0 + 0.1 \times 2.0 = 0.2 \tag{19}$$

$$\sigma^2_{\text{running}} = (1 - 0.1) \times 1 + 0.1 \times 0.5 = 0.95 \tag{20}$$

Batch 2: $\mu_{B2} = 3.0$, $\sigma^2_{B2} = 0.8$

$$\mu_{\text{running}} = 0.9 \times 0.2 + 0.1 \times 3.0 = 0.48 \tag{21}$$

$$\sigma^2_{\text{running}} = 0.9 \times 0.95 + 0.1 \times 0.8 = 0.935 \tag{22}$$

After many batches: Running averages converge to population statistics

Inference Mode: Using Running Averages

During Inference:

Switch from batch statistics to running averages:

$$y_i = \gamma \frac{x_i - \mu_{\text{running}}}{\sqrt{\sigma^2_{\text{running}} + \epsilon}} + \beta$$

Example with single image inference:

- Input pixel value: $x = 2.5$ (single activation)
- Running stats: $\mu_{\text{running}} = 3.0$, $\sigma^2_{\text{running}} = 0.675$
- Learned params: $\gamma = 2.0$, $\beta = 0.5$

Inference Mode: Using Running Averages

During Inference:

Switch from batch statistics to running averages:

$$y_i = \gamma \frac{x_i - \mu_{\text{running}}}{\sqrt{\sigma^2_{\text{running}} + \epsilon}} + \beta$$

Example with single image inference:

- Input pixel value: $x = 2.5$ (single activation)
- Running stats: $\mu_{\text{running}} = 3.0$, $\sigma^2_{\text{running}} = 0.675$
- Learned params: $\gamma = 2.0$, $\beta = 0.5$

Calculation:

$$y = 2.0 \times \left(\frac{2.5 - 3.0}{\sqrt{0.675}}\right) + 0.5 = -0.716 \tag{23}$$

Inference Mode: Using Running Averages

During Inference:

Switch from batch statistics to running averages:

$$y_i = \gamma \frac{x_i - \mu_{\text{running}}}{\sqrt{\sigma^2_{\text{running}} + \epsilon}} + \beta$$

Example with single image inference:

- Input pixel value: $x = 2.5$ (single activation)
- Running stats: $\mu_{\text{running}} = 3.0$, $\sigma^2_{\text{running}} = 0.675$
- Learned params: $\gamma = 2.0$, $\beta = 0.5$

Calculation:

$$y = 2.0 \times \left(\frac{2.5 - 3.0}{\sqrt{0.675}}\right) + 0.5 = -0.716 \tag{23}$$

Result: Stable normalization regardless of batch size!

Implementation Details

Practical Considerations:

- **Momentum value:** Typically $\alpha = 0.1$ or 0.01
  - Higher α: Faster adaptation, more noise
  - Lower α: Slower adaptation, more stable

Implementation Details

Practical Considerations:

- **Momentum value:** Typically $\alpha = 0.1$ or 0.01
  - Higher α: Faster adaptation, more noise
  - Lower α: Slower adaptation, more stable
- **Initialization:** Often start with $\mu_{\text{running}} = 0$, $\sigma^2_{\text{running}} = 1$

Implementation Details

Practical Considerations:

- **Momentum value:** Typically $\alpha = 0.1$ or 0.01
  - Higher α: Faster adaptation, more noise
  - Lower α: Slower adaptation, more stable
- **Initialization:** Often start with $\mu_{\text{running}} = 0$, $\sigma^2_{\text{running}} = 1$
- **Bias correction:** Some implementations apply correction in early training:

$$\hat{\mu}_{\text{running}} = \frac{\mu_{\text{running}}}{1 - (1 - \alpha)^t}$$

where t is the training step number

Dropout Layer

Dropout is a powerful **regularization** technique used to prevent overfitting in neural networks.

Dropout Layer

Dropout is a powerful **regularization** technique used to prevent overfitting in neural networks.

How It Works

- During each training step, Dropout randomly sets a fraction of neuron activations in a layer to **zero**.

Dropout Layer

Dropout is a powerful **regularization** technique used to prevent overfitting in neural networks.

How It Works

- During each training step, Dropout randomly sets a fraction of neuron activations in a layer to **zero**.
- This forces the network to learn more robust and redundant features, as it cannot rely on any single neuron being present.

Dropout Layer

Dropout is a powerful **regularization** technique used to prevent overfitting in neural networks.

How It Works

- During each training step, Dropout randomly sets a fraction of neuron activations in a layer to **zero**.
- This forces the network to learn more robust and redundant features, as it cannot rely on any single neuron being present.
- It prevents complex **co-adaptations**, where neurons become highly dependent on each other.

Dropout Layer

Dropout is a powerful **regularization** technique used to prevent overfitting in neural networks.

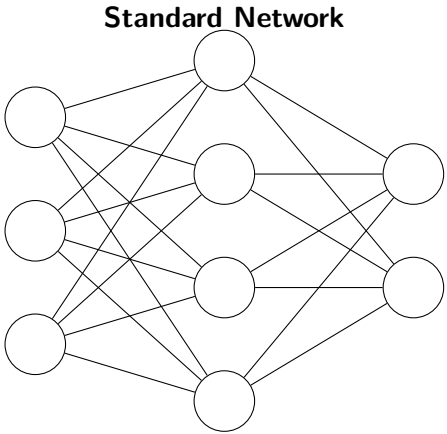
How It Works

- During each training step, Dropout randomly sets a fraction of neuron activations in a layer to **zero**.
- This forces the network to learn more robust and redundant features, as it cannot rely on any single neuron being present.
- It prevents complex **co-adaptations**, where neurons become highly dependent on each other.

Important Note

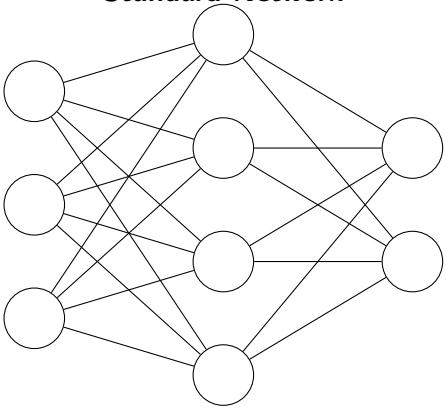
Dropout is **only applied during training**. During testing or inference, all neurons are used, but their outputs are scaled down to account for the fact that more neurons are active.

Dropout Layer Illustration

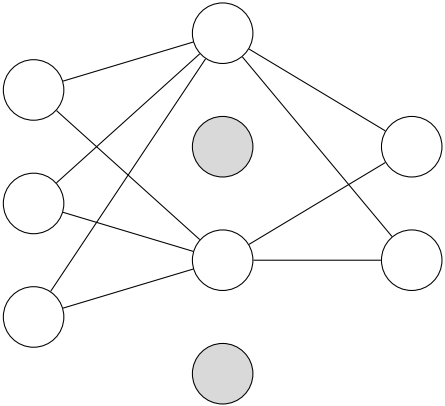


Dropout Layer Illustration

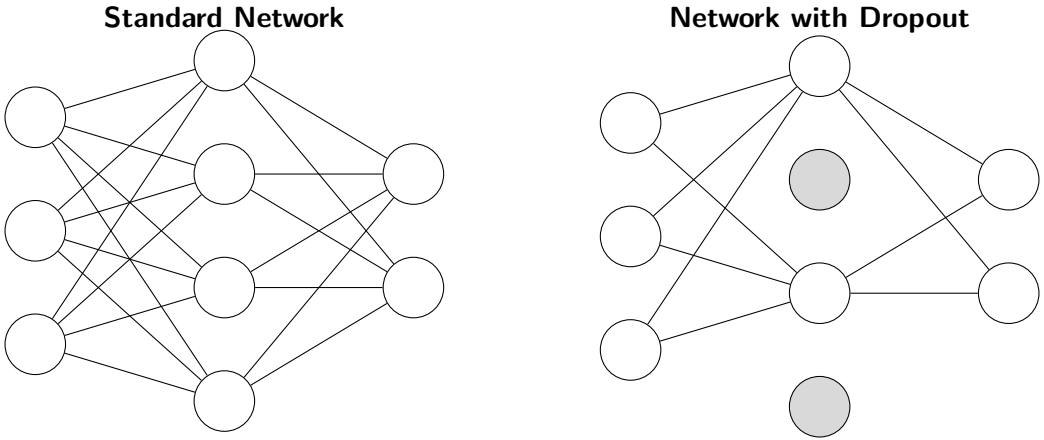
Standard Network



Network with Dropout



Dropout Layer Illustration



During training, some neurons are randomly ignored, forcing others to learn more robustly.

Dropout: Training vs Inference Modes

Training Mode

- **Random Masking:** Each neuron has probability p of being dropped
- **Scaling:** Remaining activations are scaled by $\frac{1}{1-p}$
- **Purpose:** Force network to learn robust features

Mathematical Formula:

$$y_i = \begin{cases} \frac{x_i}{1-p} & \text{with probability } (1-p) \\ 0 & \text{with probability } p \end{cases}$$

Dropout: Training vs Inference Modes

Training Mode

- **Random Masking:** Each neuron has probability p of being dropped
- **Scaling:** Remaining activations are scaled by $\frac{1}{1-p}$
- **Purpose:** Force network to learn robust features

Mathematical Formula:

$$y_i = \begin{cases} \frac{x_i}{1-p} & \text{with probability } (1-p) \\ 0 & \text{with probability } p \end{cases}$$

Inference Mode

- **No Masking:** All neurons are active
- **No Scaling:** Use original activations
- **Purpose:** Use full network capacity for prediction

Mathematical Formula:

$$y_i = x_i$$

Dropout: Training vs Inference modes

Training Mode Example (p=0.5):

- Input: $[2.0, 4.0, 1.0, 3.0]$
- Mask: $[1, 0, 1, 0]$ (random)
- Output: $[4.0, 0, 2.0, 0]$
- (Active neurons scaled by $\frac{1}{0.5} = 2$)

Inference Mode Example (same input):

- Input: $[2.0, 4.0, 1.0, 3.0]$
- Mask: $[1, 1, 1, 1]$ (all active)
- Output: $[2.0, 4.0, 1.0, 3.0]$
- (No scaling applied)

Dropout: Training vs Inference modes

Training Mode Example (p=0.5):

- Input: $[2.0, 4.0, 1.0, 3.0]$
- Mask: $[1, 0, 1, 0]$ (random)
- Output: $[4.0, 0, 2.0, 0]$
- (Active neurons scaled by $\frac{1}{0.5} = 2$)

Inference Mode Example (same input):

- Input: $[2.0, 4.0, 1.0, 3.0]$
- Mask: $[1, 1, 1, 1]$ (all active)
- Output: $[2.0, 4.0, 1.0, 3.0]$
- (No scaling applied)

Key Insight: Expected Value Consistency

The scaling in training mode ensures that the expected output remains the same:

$$\mathbb{E}[y_i^{\text{train}}] = \mathbb{E}[y_i^{\text{inference}}] = x_i$$

Training: $\mathbb{E}[y_i] = (1-p) \cdot \frac{x_i}{1-p} + p \cdot 0 = x_i$

Use of CNNs for specific Vision Task

Understanding a Typical CNN workflow

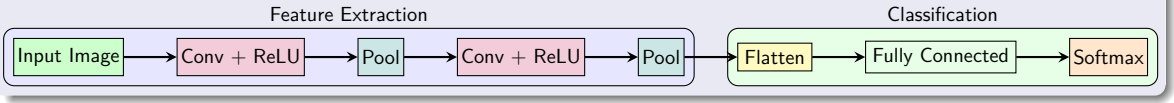
A Typical CNN Architecture

CNNs are built by stacking many layers together. A common pattern is a sequence of Convolutional, Activation, Pooling layers, and a fully connected layer for final classification.

A Typical CNN Architecture

CNNs are built by stacking many layers together. A common pattern is a sequence of Convolutional, Activation, Pooling layers, and a fully connected layer for final classification.

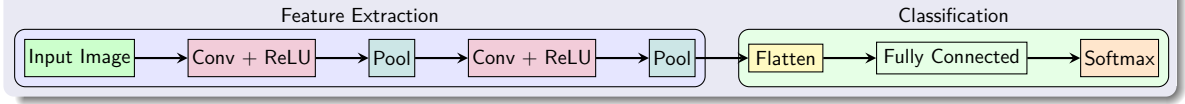
The Full Pipeline



A Typical CNN Architecture

CNNs are built by stacking many layers together. A common pattern is a sequence of Convolutional, Activation, Pooling layers, and a fully connected layer for final classification.

The Full Pipeline

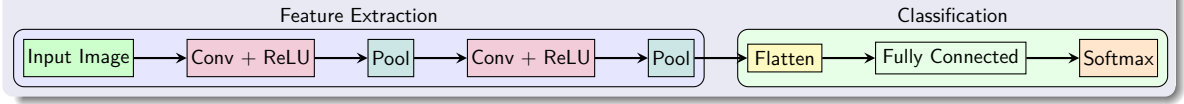


- **Convolutional Base:** The initial layers ([Conv $\rightarrow$ ReLU $\rightarrow$ Pool]) act as a **feature extractor**. They learn to turn the raw pixel data into higher-level representations. The feature maps get smaller spatially but deeper (more channels) as we go through the network.

A Typical CNN Architecture

CNNs are built by stacking many layers together. A common pattern is a sequence of Convolutional, Activation, Pooling layers, and a fully connected layer for final classification.

The Full Pipeline

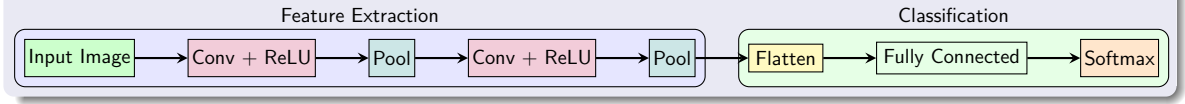


- **Convolutional Base:** The initial layers ([Conv $\rightarrow$ ReLU $\rightarrow$ Pool]) act as a **feature extractor**. They learn to turn the raw pixel data into higher-level representations. The feature maps get smaller spatially but deeper (more channels) as we go through the network.
- **Flatten Layer:** This is the bridge between the convolutional base and the classifier. It unrolls the final 3D feature maps into a 1D vector.

A Typical CNN Architecture

CNNs are built by stacking many layers together. A common pattern is a sequence of Convolutional, Activation, Pooling layers, and a fully connected layer for final classification.

The Full Pipeline



- **Convolutional Base:** The initial layers ([Conv $\rightarrow$ ReLU $\rightarrow$ Pool]) act as a **feature extractor**. They learn to turn the raw pixel data into higher-level representations. The feature maps get smaller spatially but deeper (more channels) as we go through the network.
- **Flatten Layer:** This is the bridge between the convolutional base and the classifier. It unrolls the final 3D feature maps into a 1D vector.
- **Classifier Head:** The final fully connected (dense) layers take the high-level features and perform the final classification, just like a standard MLP.

Importance of Data Augmentation

What is Data Augmentation?

Data augmentation is a technique used to artificially increase the size and diversity of a training dataset, by applying some transformations

Importance of Data Augmentation

What is Data Augmentation?

Data augmentation is a technique used to artificially increase the size and diversity of a training dataset, by applying some transformations

Why is Augmentation needed for vision tasks?

- **To Prevent Overfitting:** CNNs have millions of parameters and can easily "memorize" a small dataset. Augmentation provides more varied examples, forcing the model to learn robust and generalizable features.

Importance of Data Augmentation

What is Data Augmentation?

Data augmentation is a technique used to artificially increase the size and diversity of a training dataset, by applying some transformations

Why is Augmentation needed for vision tasks?

- **To Prevent Overfitting:** CNNs have millions of parameters and can easily "memorize" a small dataset. Augmentation provides more varied examples, forcing the model to learn robust and generalizable features.
- **To Teach Invariance:** It teaches the model that an object's identity does not change with its position, scale, or orientation. The model learns that a flipped cat is still a cat.

Importance of Data Augmentation

What is Data Augmentation?

Data augmentation is a technique used to artificially increase the size and diversity of a training dataset, by applying some transformations

Why is Augmentation needed for vision tasks?

- **To Prevent Overfitting:** CNNs have millions of parameters and can easily "memorize" a small dataset. Augmentation provides more varied examples, forcing the model to learn robust and generalizable features.
- **To Teach Invariance:** It teaches the model that an object's identity does not change with its position, scale, or orientation. The model learns that a flipped cat is still a cat.
- **To Combat Data Scarcity:** High-quality, labeled image data is often expensive and time-consuming to collect. Augmentation is a cost-effective way to maximize the value of a limited dataset.

Importance of Data Augmentation

Common Augmentation Techniques

Operations include: random rotations, horizontal/vertical flips, scaling & zooming, random cropping, and color jitter (adjusting brightness, contrast, and saturation).

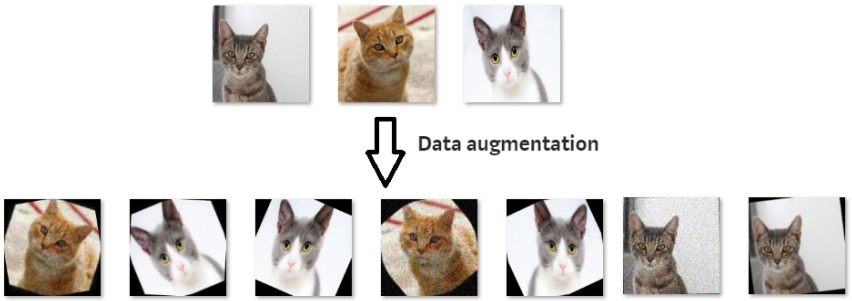


Figure: An original images and their augmented view(source: medium)

Using a CNN for Clothes Classification

Task and Dataset

- We'll apply CNNs to classify clothing items using the **Fashion-MNIST dataset**

Using a CNN for Clothes Classification

Task and Dataset

- We'll apply CNNs to classify clothing items using the **Fashion-MNIST dataset**
- Contains 60,000 training and 10,000 test grayscale images (28×28 pixels)

Using a CNN for Clothes Classification

Task and Dataset

- We'll apply CNNs to classify clothing items using the **Fashion-MNIST dataset**
- Contains 60,000 training and 10,000 test grayscale images (28×28 pixels)
- 10 clothing categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot

Using a CNN for Clothes Classification

Task and Dataset

- We'll apply CNNs to classify clothing items using the **Fashion-MNIST dataset**
- Contains 60,000 training and 10,000 test grayscale images (28×28 pixels)
- 10 clothing categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot

CNN Architecture:

- 3 Convolutional blocks (Conv + ReLU + MaxPool)
- Filter progression: $32 \rightarrow 64 \rightarrow 128$ channels
- Each conv layer uses 3×3 kernels
- 2×2 max pooling reduces spatial dimensions

Classification Head:

- Flatten $3 \times 3 \times 128$ feature maps
- Fully connected layer ($1152 \rightarrow 128$)
- Dropout for regularization
- Output layer ($128 \rightarrow 10$ classes)

CNN Data Flow and Feature Learning

Data Flow Through the Network:

Layer	Operation	Output Shape
Input	Raw image	[1, 28, 28]
Conv1 + Pool1	32 filters, 2×2 pool	[32, 14, 14]
Conv2 + Pool2	64 filters, 2×2 pool	[64, 7, 7]
Conv3 + Pool3	128 filters, 2×2 pool	[128, 3, 3]
Flatten	Reshape to 1D	[1152]
FC + Dropout	Dense layer	[128]
Output	Classification	[10]

Hierarchical Feature Learning:

- **Early layers (Conv1):** Detect edges, lines, basic patterns
- **Middle layers (Conv2):** Combine edges into textures, shapes
- **Deep layers (Conv3):** Recognize clothing parts, complex patterns
- **FC layers:** Combine features for final classification decision

Training Process and Results

Training Setup:

- Dataset: Fashion-MNIST (60,000 training, 10,000 test images)
- Preprocessing: Normalize pixel values to [0,1] range
- Batch size: 64 samples per batch
- Training epochs: 5 (demonstration purposes)

Model Performance:

- Total parameters: 100K (much fewer than equivalent MLP)
- Training accuracy: Improves progressively each epoch
- Test accuracy: 85-90% (varies with training duration)

Famous CNN Architectures

State Of The Art CNN Architectures

The ImageNet Challenge

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) was an annual competition that became the catalyst for the modern deep learning revolution since 2006.

The ImageNet Challenge

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) was an annual competition that became the catalyst for the modern deep learning revolution since 2006.

The Dataset & Task

- **Dataset:** A massive collection of over 14 million hand-labeled images across more than 20,000 categories.

The ImageNet Challenge

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) was an annual competition that became the catalyst for the modern deep learning revolution since 2006.

The Dataset & Task

- **Dataset:** A massive collection of over 14 million hand-labeled images across more than 20,000 categories.
- **Task:** The main challenge involved classifying an image into one of 1,000 distinct categories.

The ImageNet Challenge

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) was an annual competition that became the catalyst for the modern deep learning revolution since 2006.

The Dataset & Task

- **Dataset:** A massive collection of over 14 million hand-labeled images across more than 20,000 categories.
- **Task:** The main challenge involved classifying an image into one of 1,000 distinct categories.
- **Metric:** Performance was measured by the **top-5 error rate** – a prediction was only considered incorrect if the true label was not among the model's top five guesses.

The ImageNet Challenge

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) was an annual competition that became the catalyst for the modern deep learning revolution since 2006.

The Dataset & Task

- **Dataset:** A massive collection of over 14 million hand-labeled images across more than 20,000 categories.
- **Task:** The main challenge involved classifying an image into one of 1,000 distinct categories.
- **Metric:** Performance was measured by the **top-5 error rate** – a prediction was only considered incorrect if the true label was not among the model's top five guesses.

Legacy:

The competition's success proved that deep learning, powered by large datasets and GPUs, was the key to solving complex computer vision problems, hence enabling development of many State-Of-The-Art CNN architectures.

AlexNet

Key Details

- A much deeper convolutional network that won the ImageNet challenge in 2012.

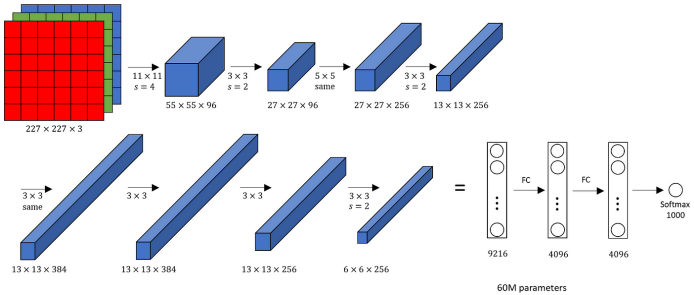


Figure: AlexNet architecture. (source: medium)

AlexNet

Key Details

- A much deeper convolutional network that won the ImageNet challenge in 2012.
- Introduced the use of ReLU activations.

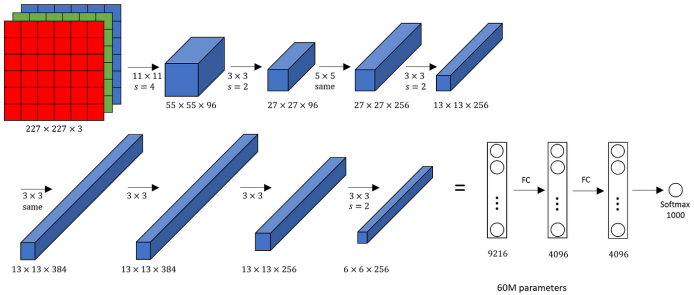


Figure: AlexNet architecture. (source: medium)

AlexNet

Key Details

- A much deeper convolutional network that won the ImageNet challenge in 2012.
- Introduced the use of ReLU activations.
- Intoduced heavy GPU parallelism.

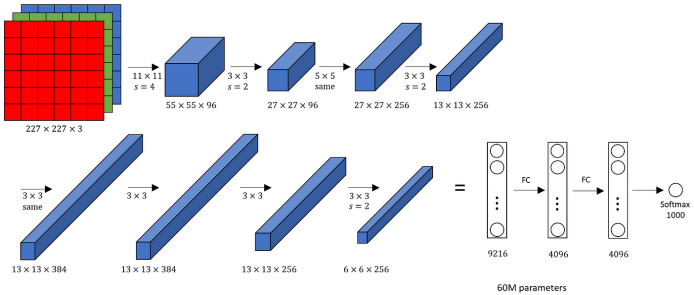


Figure: AlexNet architecture. (source: medium)

VGGNet

Key Details

- **Homogeneous Architecture:** Proved that extreme network depth (16-19 layers) was crucial for performance, setting a new standard.

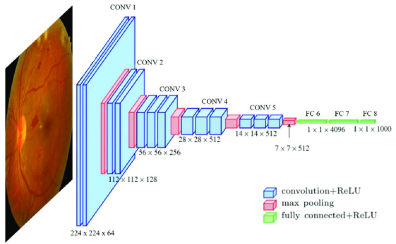


Figure: VGG16 architecture. (source: medium)

VGGNet

Key Details

- **Homogeneous Architecture:** Proved that extreme network depth (16-19 layers) was crucial for performance, setting a new standard.
- **Exclusive 3×3 Kernels:** Used only very small 3×3 convolutional filters stacked on top of each other. A stack of three 3×3 layers has the same effective receptive field as one 7×7 layer, but with more non-linearity and fewer parameters.

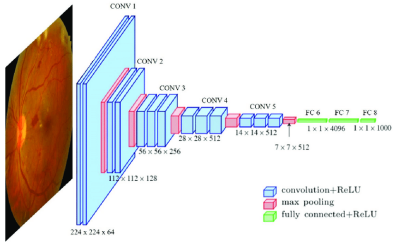


Figure: VGG16 architecture. (source: medium)

ResNet

Key Details

- ResNet introduced "residual blocks" that feature a skip connection, adding the block's input x directly to its output $F(x)$. This forces the layer to learn a residual function $F(x) = H(x) - x$ instead of the entire mapping $H(x)$.

ResNet

Key Details

- ResNet introduced "residual blocks" that feature a skip connection, adding the block's input x directly to its output $F(x)$. This forces the layer to learn a residual function $F(x) = H(x) - x$ instead of the entire mapping $H(x)$.
- This framework makes it trivial for layers to learn an identity function simply by driving the weights of $F(x)$ to zero. This ensures that adding a new layer will, at worst, do nothing, and will not degrade performance.

ResNet

Key Details

- ResNet introduced "residual blocks" that feature a skip connection, adding the block's input x directly to its output $F(x)$. This forces the layer to learn a residual function $F(x) = H(x) - x$ instead of the entire mapping $H(x)$.
- This framework makes it trivial for layers to learn an identity function simply by driving the weights of $F(x)$ to zero. This ensures that adding a new layer will, at worst, do nothing, and will not degrade performance.
- Residual learning solved the degradation problem and allowed for the training of extremely deep networks (e.g., 101 or 152 layers) without sacrificing performance.

ResNet: Solving the Degradation Problem

Why Residual Blocks?

- Researchers found that as network depth increased, accuracy would saturate and then begin to degrade rapidly. A 56-layer network often performed worse than a 20-layer one.

ResNet: Solving the Degradation Problem

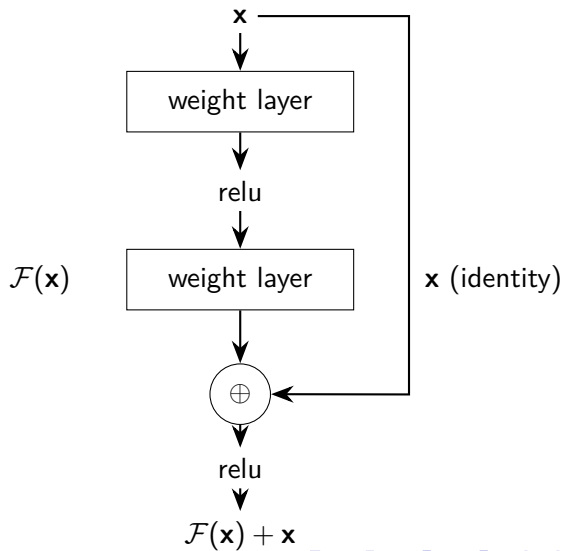
Why Residual Blocks?

- Researchers found that as network depth increased, accuracy would saturate and then begin to degrade rapidly. A 56-layer network often performed worse than a 20-layer one.
- The network struggled to even learn a simple identity mapping (where extra layers just pass the input through).

ResNet: Solving the Degradation Problem

Why Residual Blocks?

- Researchers found that as network depth increased, accuracy would saturate and then begin to degrade rapidly. A 56-layer network often performed worse than a 20-layer one.
- The network struggled to even learn a simple identity mapping (where extra layers just pass the input through).
- **Skip connections** explicitly provide this identity path, allowing the optimizer to only focus on learning the useful residual, thus solving the degradation puzzle.



ResNet Architecture

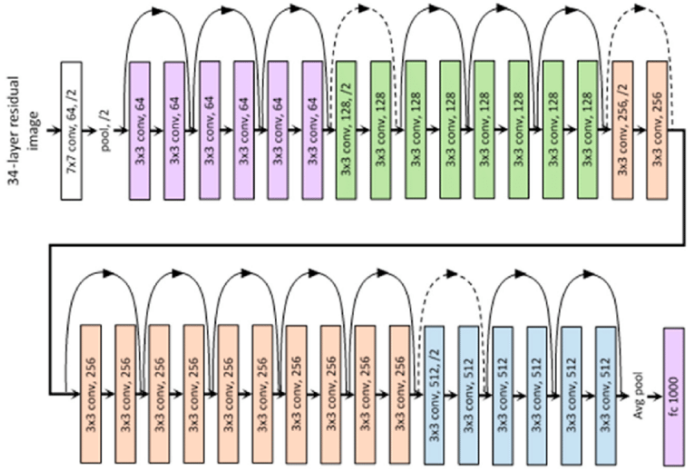


Figure: ResNet Architecture. (source: medium)

GoogleNet

Key Details

- **The Inception Module:** It performs parallel convolutions (1×1, 3×3, 5×5) and pooling on the same input, then concatenates their feature maps.

GoogleNet

Key Details

- **The Inception Module:** It performs parallel convolutions (1×1, 3×3, 5×5) and pooling on the same input, then concatenates their feature maps.
- 1×1 **Bottleneck:** To make the parallel convolutions computationally feasible, it masterfully uses 1×1 convolutions as a "bottleneck" layer to perform dimensionality reduction **before** the expensive 3×3 and 5×5 convolutions.

GoogleNet

Key Details

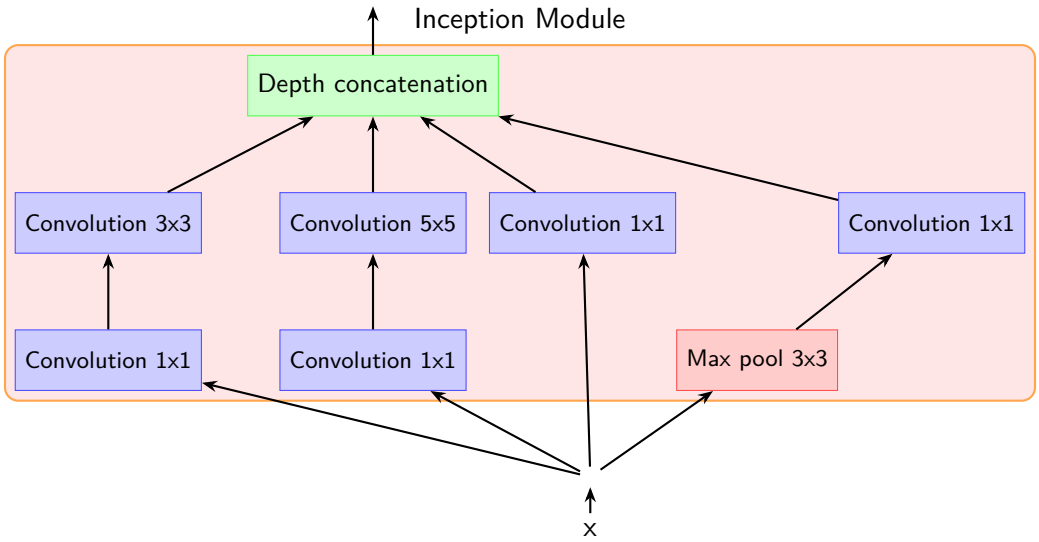
- **The Inception Module:** It performs parallel convolutions (1×1, 3×3, 5×5) and pooling on the same input, then concatenates their feature maps.
- 1×1 **Bottleneck:** To make the parallel convolutions computationally feasible, it masterfully uses 1×1 convolutions as a "bottleneck" layer to perform dimensionality reduction **before** the expensive 3×3 and 5×5 convolutions.
- **Computationally Efficient:** Achieved state-of-the-art results with a much lower parameter count than VGG.

GoogleNet

Key Details

- **The Inception Module:** It performs parallel convolutions (1×1, 3×3, 5×5) and pooling on the same input, then concatenates their feature maps.
- 1×1 **Bottleneck:** To make the parallel convolutions computationally feasible, it masterfully uses 1×1 convolutions as a "bottleneck" layer to perform dimensionality reduction **before** the expensive 3×3 and 5×5 convolutions.
- **Computationally Efficient:** Achieved state-of-the-art results with a much lower parameter count than VGG.
- It replaced the final fully connected layers with a single **Global Average Pooling (GAP)** layer, further reducing parameters.

The Inception Module



GoogleNet Architecture

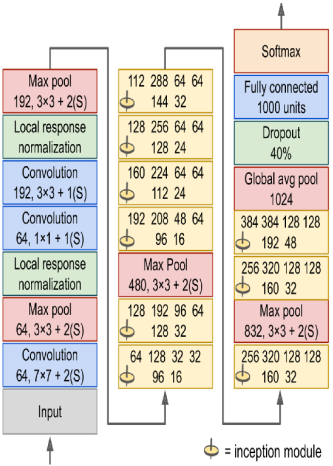


Figure: GoogleNet Architecture (source: ISBN:978-1-4920-3264-9)

Transfer Learning with CNNs

Utilizing the famous Architectures for specific tasks

Introduction to Transfer Learning

Don't Reinvent the Wheel

Training a large CNN from scratch requires massive datasets (like ImageNet) and huge computational resources.

Introduction to Transfer Learning

Don't Reinvent the Wheel

Training a large CNN from scratch requires massive datasets (like ImageNet) and huge computational resources.

Transfer Learning is the process of taking a model pre-trained on a large dataset and fine-tuning it for a new, specific task.

Introduction to Transfer Learning

Don't Reinvent the Wheel

Training a large CNN from scratch requires massive datasets (like ImageNet) and huge computational resources.

Transfer Learning is the process of taking a model pre-trained on a large dataset and fine-tuning it for a new, specific task.

1. **Use the pre-trained convolutional base** as a fixed feature extractor.

Introduction to Transfer Learning

Don't Reinvent the Wheel

Training a large CNN from scratch requires massive datasets (like ImageNet) and huge computational resources.

Transfer Learning is the process of taking a model pre-trained on a large dataset and fine-tuning it for a new, specific task.

1. **Use the pre-trained convolutional base** as a fixed feature extractor.
2. **Remove the original classifier head.**

Introduction to Transfer Learning

Don't Reinvent the Wheel

Training a large CNN from scratch requires massive datasets (like ImageNet) and huge computational resources.

Transfer Learning is the process of taking a model pre-trained on a large dataset and fine-tuning it for a new, specific task.

1. **Use the pre-trained convolutional base** as a fixed feature extractor.
2. **Remove the original classifier head.**
3. **Add a new, small classifier head** for your specific task.

Introduction to Transfer Learning

Don't Reinvent the Wheel

Training a large CNN from scratch requires massive datasets (like ImageNet) and huge computational resources.

Transfer Learning is the process of taking a model pre-trained on a large dataset and fine-tuning it for a new, specific task.

1. **Use the pre-trained convolutional base** as a fixed feature extractor.
2. **Remove the original classifier head.**
3. **Add a new, small classifier head** for your specific task.
4. **Train only the new classifier head** on your smaller, custom dataset.

This is the standard and most effective approach for most practical computer vision problems.

Various forms of Transfer Learning

Transfer learning is a strategy where a model trained on one large task is repurposed as the starting point for a different, related task.

Various forms of Transfer Learning

Transfer learning is a strategy where a model trained on one large task is repurposed as the starting point for a different, related task.

As a Feature Extractor

- The pre-trained convolutional base (e.g., VGG, ResNet) is used as a fixed feature extractor.

Various forms of Transfer Learning

Transfer learning is a strategy where a model trained on one large task is repurposed as the starting point for a different, related task.

As a Feature Extractor

- The pre-trained convolutional base (e.g., VGG, ResNet) is used as a fixed feature extractor.
- We "freeze" the weights of the base layers, remove the original classifier head, and add a new classifier.

Various forms of Transfer Learning

Transfer learning is a strategy where a model trained on one large task is repurposed as the starting point for a different, related task.

As a Feature Extractor

- The pre-trained convolutional base (e.g., VGG, ResNet) is used as a fixed feature extractor.
- We "freeze" the weights of the base layers, remove the original classifier head, and add a new classifier.
- Only the new classifier is trained. This is ideal when your new dataset is very small.

Various forms of Transfer Learning

Transfer learning is a strategy where a model trained on one large task is repurposed as the starting point for a different, related task.

As a Feature Extractor

- The pre-trained convolutional base (e.g., VGG, ResNet) is used as a fixed feature extractor.
- We "freeze" the weights of the base layers, remove the original classifier head, and add a new classifier.
- Only the new classifier is trained. This is ideal when your new dataset is very small.

As a Fine-Tuner

- We initialize the network with pre-trained weights, just like before.

Various forms of Transfer Learning

Transfer learning is a strategy where a model trained on one large task is repurposed as the starting point for a different, related task.

As a Feature Extractor

- The pre-trained convolutional base (e.g., VGG, ResNet) is used as a fixed feature extractor.
- We "freeze" the weights of the base layers, remove the original classifier head, and add a new classifier.
- Only the new classifier is trained. This is ideal when your new dataset is very small.

As a Fine-Tuner

- We initialize the network with pre-trained weights, just like before.
- We "unfreeze" either the entire network or the top few convolutional layers.

Various forms of Transfer Learning

Transfer learning is a strategy where a model trained on one large task is repurposed as the starting point for a different, related task.

As a Feature Extractor

- The pre-trained convolutional base (e.g., VGG, ResNet) is used as a fixed feature extractor.
- We "freeze" the weights of the base layers, remove the original classifier head, and add a new classifier.
- Only the new classifier is trained. This is ideal when your new dataset is very small.

As a Fine-Tuner

- We initialize the network with pre-trained weights, just like before.
- We "unfreeze" either the entire network or the top few convolutional layers.
- We continue training the whole model (or part of it) end-to-end on the new data using a very low learning rate.

Various forms of Transfer Learning

Transfer learning is a strategy where a model trained on one large task is repurposed as the starting point for a different, related task.

As a Feature Extractor

- The pre-trained convolutional base (e.g., VGG, ResNet) is used as a fixed feature extractor.
- We "freeze" the weights of the base layers, remove the original classifier head, and add a new classifier.
- Only the new classifier is trained. This is ideal when your new dataset is very small.

As a Fine-Tuner

- We initialize the network with pre-trained weights, just like before.
- We "unfreeze" either the entire network or the top few convolutional layers.
- We continue training the whole model (or part of it) end-to-end on the new data using a very low learning rate.
- This is effective when you have a larger dataset.

Transfer Learning for Image classification

This is the most common use case for transfer learning, leveraging models trained on large-scale datasets.

- **Use a Pre-trained Backbone:** We start with a network (like ResNet50 or MobileNet) that has been pre-trained on the massive **ImageNet** dataset (1.4 million images, 1000 classes).

Transfer Learning for Image classification

This is the most common use case for transfer learning, leveraging models trained on large-scale datasets.

- **Use a Pre-trained Backbone:** We start with a network (like ResNet50 or MobileNet) that has been pre-trained on the massive **ImageNet** dataset (1.4 million images, 1000 classes).
- **Why it works:** This pre-trained model has already learned a powerful hierarchy of visual features, from universal edges and textures in early layers to complex object parts in deeper layers. These features are useful for almost any visual recognition task.

Transfer Learning for Image classification

This is the most common use case for transfer learning, leveraging models trained on large-scale datasets.

- **Use a Pre-trained Backbone:** We start with a network (like ResNet50 or MobileNet) that has been pre-trained on the massive **ImageNet** dataset (1.4 million images, 1000 classes).
- **Why it works:** This pre-trained model has already learned a powerful hierarchy of visual features, from universal edges and textures in early layers to complex object parts in deeper layers. These features are useful for almost any visual recognition task.
- **How it's done:** We remove the original top classification layer (the one that predicts 1000 ImageNet classes) and replace it with a new, randomly initialized classifier that matches our specific number of classes (e.g., 2 classes for "cat vs. dog"). This new model is then fine-tuned on our smaller dataset.

Transfer Learning for Segmentation

For segmentation (per-pixel classification), transfer learning is used to build the **encoder** part of an encoder-decoder network.

- **The Architecture:** Modern segmentation models like **U-Net** or FCN (Fully Convolutional Network) consist of an *encoder* path (which downsamples) and a *decoder* path (which upsamples).

Transfer Learning for Segmentation

For segmentation (per-pixel classification), transfer learning is used to build the **encoder** part of an encoder-decoder network.

- **The Architecture:** Modern segmentation models like **U-Net** or FCN (Fully Convolutional Network) consist of an *encoder* path (which downsamples) and a *decoder* path (which upsamples).
- **Using the Pre-trained Model:** The entire encoder path is replaced with a pre-trained classification network (like a ResNet or VGG base, stripped of its final classifier). This is called the "backbone." This backbone provides the rich semantic feature extraction ("what" is in the image) learned from ImageNet.

Transfer Learning for Segmentation

For segmentation (per-pixel classification), transfer learning is used to build the **encoder** part of an encoder-decoder network.

- **The Architecture:** Modern segmentation models like **U-Net** or FCN (Fully Convolutional Network) consist of an *encoder* path (which downsamples) and a *decoder* path (which upsamples).
- **Using the Pre-trained Model:** The entire encoder path is replaced with a pre-trained classification network (like a ResNet or VGG base, stripped of its final classifier). This is called the "backbone." This backbone provides the rich semantic feature extraction ("what" is in the image) learned from ImageNet.
- **Training the Decoder:** A new decoder "head" is added. This decoder takes the powerful, low-resolution features from the encoder and progressively upsamples them (often using skip connections) to rebuild the high-resolution, pixel-level segmentation mask. Only this decoder (or the whole fine-tuned network) is trained on the segmentation dataset.